

Java 25 — Module Import Declarations

Java Version: 25

Year: 2025

Feature: Module Import Declarations

UniqueClassName: ModuleImportDeclarations

JEP: 511

Status: Final

1. Definition

Module Import Declarations is a Java 25 language feature that allows a Java source file to **import all the accessible packages exported by a module** with a single declaration.

The syntax is:

```
import module java.base;
```

Instead of writing many individual imports:

```
import java.util.List;
import java.util.ArrayList;
import java.util.Map;
import java.util.HashMap;
import java.util.Optional;
```

you can use:

```
import module java.base;
```

This is particularly useful for **small programs, educational examples, scripts, and beginners**, where determining the correct package for every class can otherwise be distracting.

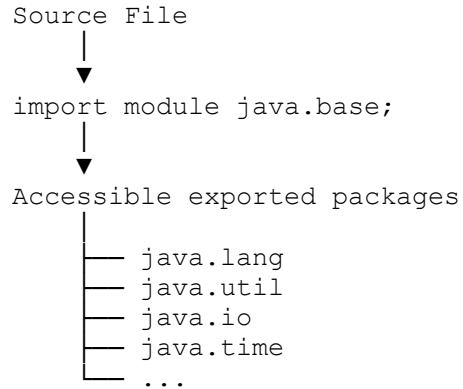
2. Basic Concept

Traditional imports work at the **package/type level**:

Source File

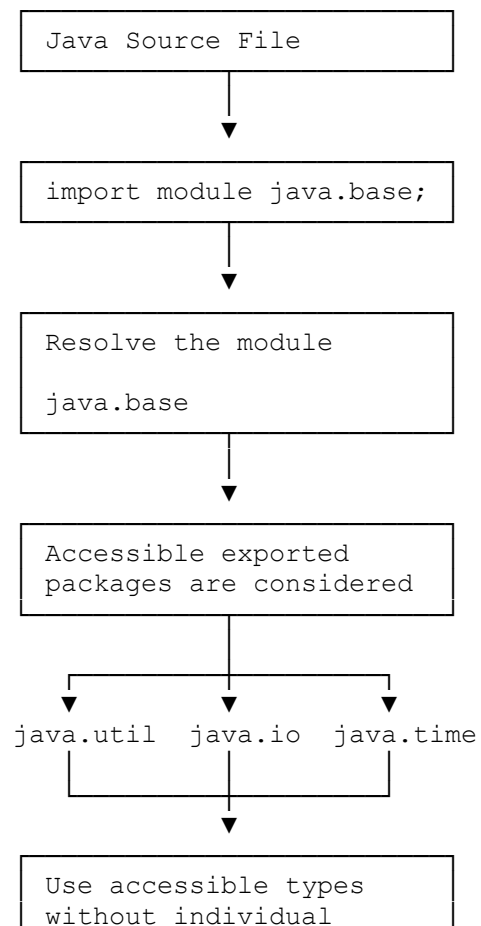
```
|
|— import java.util.List
|— import java.util.Map
|— import java.util.Optional
```

Module import declarations work at the **module level**:



The module declaration therefore acts as a convenient way to bring the module's accessible exported types into scope.

3. Flowchart



```
package imports
```

4. Java Code Example

Here is a simple Java 25 example:

```
import module java.base;

public class ModuleImportDeclarations {

    public static void main(String[] args) {

        List<String> names =
            new ArrayList<>();

        names.add("Anurag");
        names.add("Rahul");
        names.add("Amit");

        for (String name : names) {
            System.out.println(name);
        }
    }
}
```

Notice that we did **not** write:

```
import java.util.List;
import java.util.ArrayList;
```

Instead:

```
import module java.base;
```

provides the module-level import.

Output

```
Anurag
Rahul
Amit
```

5. Another Example with Multiple Packages

Because java.base contains many commonly used packages, you can use types from different packages.

```
import module java.base;

public class ModuleImportDeclarations {

    public static void main(String[] args) {

        List<Integer> numbers =
            List.of(10, 20, 30, 40);

        System.out.println(
            "Numbers: " + numbers
        );

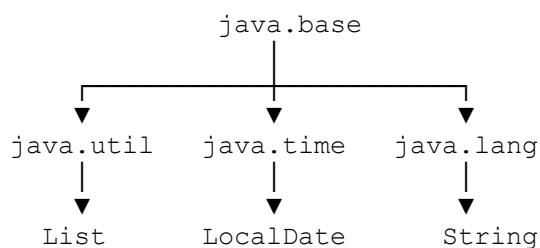
        LocalDate today =
            LocalDate.now();

        System.out.println(
            "Today: " + today
        );

        String text =
            String.join("-", "Java", "25");

        System.out.println(text);
    }
}
```

Conceptually:



6. Traditional Imports vs Module Import

Traditional approach

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
```

```
import java.util.Map;
import java.time.LocalDate;
import java.time.LocalDateTime;
```

Module import

```
import module java.base;
```

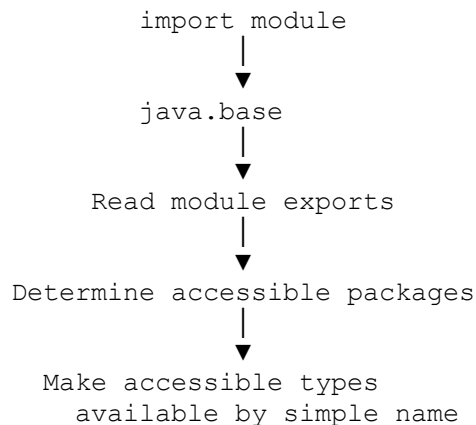
The second form can significantly reduce the number of import declarations.

7. How Module Import Works

Consider:

```
import module java.base;
```

The compiler conceptually does the following:



It is **not equivalent to importing every class blindly**. Module readability, exports, accessibility, and name-resolution rules still apply.

8. Module Import vs Wildcard Import

These two mechanisms are different.

Package wildcard

```
import java.util.*;
```

means approximately:

Import accessible types from java.util

Only **one package** is involved.

Module import

```
import module java.base;
```

means:

Consider accessible exported packages
from the specified module.

Conceptually:

```
java.util.*  
└── One package
```

```
module java.base  
├── java.lang  
├── java.util  
├── java.io  
├── java.time  
├── java.net  
└── ...
```

9. A More Interesting Example

```
import module java.base;
```

```
public class ModuleImportDeclarations {  
    public static void main(String[] args) {  
        List<String> languages =  
            new ArrayList<>();  
  
        languages.add("Java");  
        languages.add("Python");  
        languages.add("C++");  
  
        Set<String> uniqueLanguages =  
            new HashSet<>(languages);  
  
        Map<String, Integer> lengths =  
            new HashMap<>();  
  
        for (String language : uniqueLanguages) {
```

```

        lengths.put (
            language,
            language.length ()
        );
    }

    System.out.println (lengths);
}

```

Here the program uses several classes from java.util:

List
 ArrayList
 Set
 HashSet
 Map
 HashMap

Yet only one module import is needed:

```
import module java.base;
```

10. Why Was This Feature Introduced?

One important motivation is to make Java easier to use for **small programs and beginner-oriented code**.

Previously, a beginner might encounter:

```
import java.util.ArrayList;
import java.util.List;
import java.util.HashMap;
import java.util.Map;
```

and wonder:

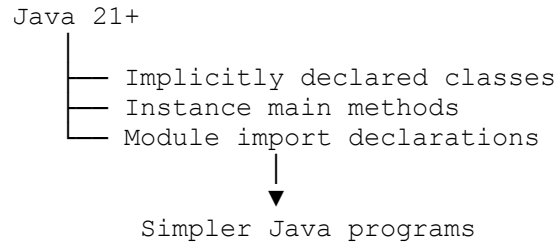
"How do I know which package contains each class?"

With module import declarations:

```
import module java.base;
```

the programmer can initially focus on the Java code rather than package organization.

This works particularly well alongside Java's other recent simplification features:

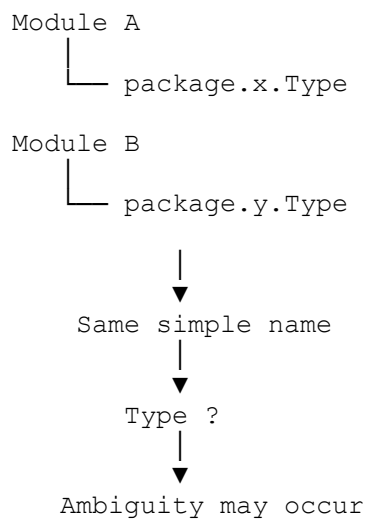


11. Name Conflicts

A module import can potentially make more simple names available.

For example, if two imported modules expose types with the same simple name, Java's normal name-resolution rules still apply.

Conceptually:



In such situations, an explicit single-type import can be used to resolve the intended type.

12. Important Difference: java.base

java.base is special because it is the foundational Java SE module.

It provides core packages such as:

```
java.lang
java.io
java.util
```

```
java.time
java.net
java.nio
java.security
...
```

Therefore:

```
import module java.base;
```

can be useful in many standalone examples.

13. Java 25 Catalog Entry

For your Java feature catalog:

Java Version	: 25
Year	: 2025
Feature	: Module Import Declarations
UniqueClassName	: ModuleImportDeclarations
JEP	: 511
Status	: Final

The feature was proposed to simplify imports by allowing an entire module's exported packages to be imported rather than requiring numerous individual package imports.

Key Takeaway

Module Import Declarations in Java 25 allow developers to write `import module module.name;` to import accessible types from the exported packages of a module. The feature reduces import boilerplate and is especially useful for small programs, teaching, and beginner-friendly Java code.

Java 25 — Compact Source Files and Instance Main Methods

Java Version: 25

Year: 2025

Feature: Compact Source Files and Instance Main Methods

UniqueClassName: CompactSourceFilesInstanceMain

JEP: 512

Status: Final

1. Definition

Compact Source Files and Instance Main Methods is a Java 25 language feature that makes it possible to write small Java programs with **much less boilerplate**.

It allows a source file to:

1. **Declare methods and fields without explicitly declaring a class**, and
2. Use an **instance main method** as the entry point.

For example, traditional Java requires:

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Java 25 allows the much simpler:

```
void main() {  
    System.out.println("Hello, World!");  
}
```

This is particularly useful for:

- beginners learning Java,
- small programs,
- teaching,
- demonstrations,
- simple scripts,
- experimentation.

2. Traditional Java vs Compact Source File

Traditional Java

Source File



```
public class Hello

    public static void
    main(String[] args)
        ↓
    application code
```

There are several pieces of syntax that a beginner must understand before writing the first program.

Compact source file

Source File



```
void main() {
    ...
}
```

The Java compiler provides the surrounding class structure implicitly.

3. Flowchart

Before (Java 17)

```
public class Hello {
    public static void
    main(String[] args)
    {
        System.out.println
        ('Hello, World!);
    }
}
```

After (Java 25)

```
class Hello
void main() {
    System.out.println
    ('Hello, World!);
}
```

Java 17 (Traditional)	Java 25 (Modern)
<pre>public class HelloWorld { public static void main(String[] args) { System.out.println('Hello, World!'); } }</pre>	<pre>void main() { IO.println('Hello, World!'); }</pre>
<pre>public class Greet public static void main(String[] arg var name = args.length > 0 ? args[0]: 'User'; } }</pre>	<pre>void main(String[] args) { var name = args.length > 0 ? args[0]: User; IO.println(Hello + name); }</pre>

Paving the On-Ramp (JEP 512)

Before

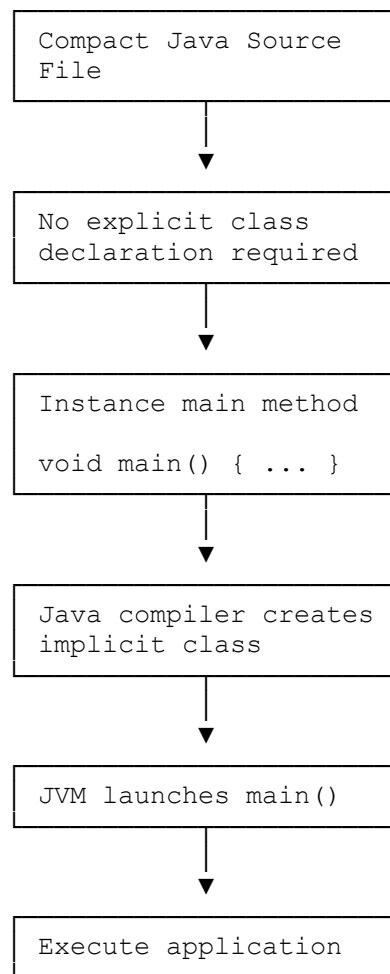
```
• public class HelloWorld {
    • public static void main(String[] args) {
        • System.out.println("Hello, World!");
    • }
• }
```

After

```
void main() {
    IO.println("Hello World!");
}
```

- Classes for modeling and organization
- Static vs instance behavior
- Command line interaction, arrays
- Magic method name
- Access control
- Static fields

openjdk.org/jeps/512



4. Simplest Java 25 Example

Create a file:

Hello.java

with:

```
void main() {  
    System.out.println("Hello, Java 25!");  
}
```

That's it.

There is no:

```
public class Hello
```

and no:

```
public static void main(String[] args)
```

Output

Hello, Java 25!

5. How It Works

The source:

```
void main() {  
    System.out.println("Hello, Java 25!");  
}
```

is conceptually treated as if Java provides a class around it:

```
class Hello {  
  
    void main() {  
        System.out.println("Hello, Java 25!");  
    }  
}
```

The class itself is **implicitly declared**.

The programmer therefore focuses directly on the application logic.

6. Instance main() Method

Traditional Java uses:

```
public static void main(String[] args)
```

Java 25 permits:

```
void main()
```

This is an **instance method**, rather than a static method.

For example:

```
void main() {  
    String message = "Welcome to Java 25";  
    System.out.println(message);  
}
```

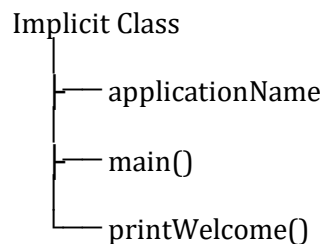
The JVM can launch the program without requiring the programmer to explicitly write static.

7. Instance Fields and Methods

Compact source files can contain fields and additional methods.

```
String applicationName = "Java 25";  
  
void main() {  
    printWelcome();  
    System.out.println(  
        "Application: " + applicationName  
    );  
}  
  
void printWelcome() {  
    System.out.println(  
        "Welcome to " + applicationName  
    );  
}
```

Conceptually:



Output

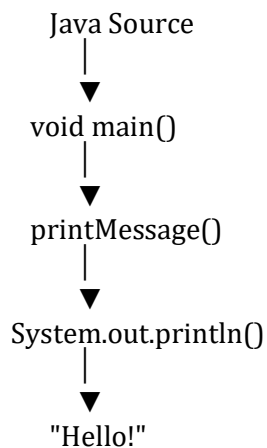
```
Welcome to Java 25  
Application: Java 25
```

8. Flow of Execution

For:

```
void main() {  
    printMessage();  
}  
  
void printMessage() {  
    System.out.println("Hello!");  
}
```

the execution flow is:



9. Using Imports

Compact source files can also use imports.

For example:

```
import java.util.List;  
  
void main() {  
    List<String> languages =  
        List.of("Java", "Python", "C++");  
  
    for (String language : languages) {  
        System.out.println(language);  
    }  
}
```

Output:

```
Java  
Python  
C++
```

And because Java 25 also provides **Module Import Declarations**, a small program can potentially use:

```
import module java.base;  
  
void main() {  
    List<String> names =  
        List.of("Anurag", "Rahul", "Amit");  
  
    names.forEach(System.out::println);  
}
```

This demonstrates how multiple Java 25 simplifications can work together.

10. main() Parameters

The compact form can use an instance main() method with command-line arguments as well.

For example:

```
void main(String[] args) {  
    System.out.println(  
        "Arguments: " + args.length  
    );  
  
    for (String arg : args) {  
        System.out.println(arg);  
    }  
}
```

So the programmer doesn't have to use the traditional:

```
public static void main(String[] args)
```

merely to access command-line arguments.

11. Comparison

Traditional Java	Compact Source File
Explicit class declaration	Class can be implicit
public static void main(String[] args)	void main()
More boilerplate	Less boilerplate
Explicit class structure	Compiler provides implicit structure
Good for larger applications	Excellent for small programs/learning
Java 1-24 style	Java 25 final feature

12. Important Restriction

This feature is primarily intended to make **small Java programs simpler**.

For a large enterprise application, you would normally still use explicit classes:

```
public class CustomerService {  
    // ...  
}
```

rather than putting everything into a compact source file.

A useful way to think about it is:

Small program



Compact Source File



Less boilerplate

Large application



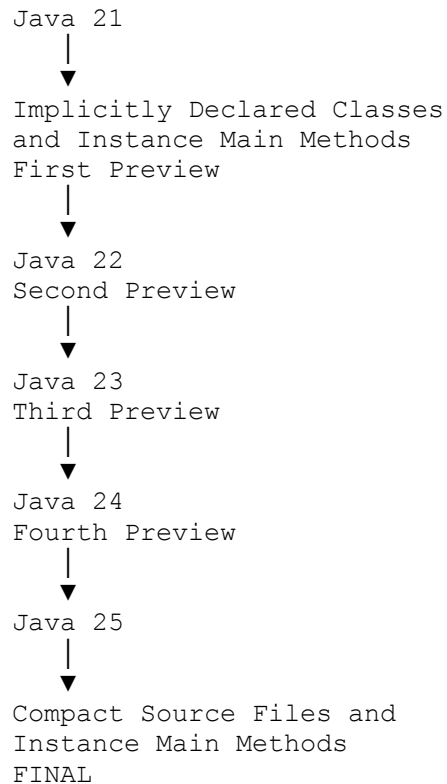
Explicit classes/packages/modules



Better organization

13. Relationship to Earlier Java Releases

This feature evolved through previews:



In Java 25, the feature was finalized under **JEP 512**.

14. Java 25 Catalog Entry

For your Java feature catalog:

Java Version	: 25
Year	: 2025
Feature	: Compact Source Files and Instance Main Methods
UniqueClassName	: CompactSourceFilesInstanceMain
JEP	: 512
Status	: Final

Key Takeaway

Compact Source Files and Instance Main Methods in Java 25 allow small Java programs to omit the explicit class declaration and use a simple instance void main() method. The result is dramatically less boilerplate while retaining the full Java language underneath.

Java 25 — Flexible Constructor Bodies

Java Version: 25

Year: 2025

Feature: Flexible Constructor Bodies

UniqueClassName: FlexibleConstructorBodies

JEP: 513

Status: Final

1. Definition

Flexible Constructor Bodies is a Java 25 language feature that allows statements to appear **before an explicit constructor invocation** such as super(...) or this(...), provided those statements do not access the object under construction in ways that are prohibited before superclass initialization.

Previously, Java required an explicit constructor invocation to be the **first statement** in a constructor:

```
class Child extends Parent {  
  
    Child(int value) {  
        super(value); // Must traditionally come first  
  
        // constructor body  
    }  
}
```

Java 25 makes constructor code more flexible:

```
class Child extends Parent {  
  
    Child(int value) {  
  
        int adjusted = value * 2;  
  
        super(adjusted);  
  
        System.out.println(adjusted);  
    }  
}
```

This is especially useful when you need to **prepare or validate constructor arguments before calling super(...)**.

2. The Problem Before Java 25

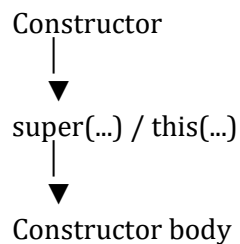
Suppose the superclass constructor requires a calculated value:

```
class Parent {  
    Parent(int value) {  
        System.out.println("Parent: " + value);  
    }  
}
```

Previously, if you wanted to calculate the argument:

```
class Child extends Parent {  
    Child(int value) {  
        int adjusted = value * 2;  
        super(adjusted); // Not allowed before Java 25  
    }  
}
```

Java's old constructor rule required:

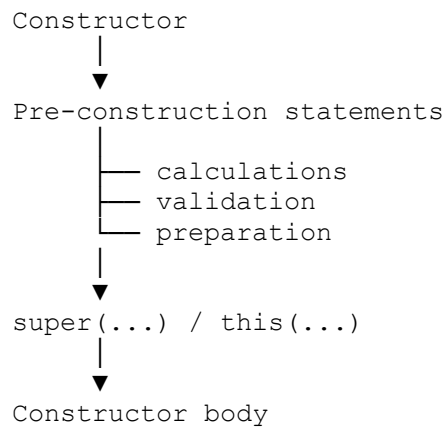


So developers often had to move calculations into:

- static methods,
 - constructor arguments,
 - temporary helper methods,
 - or complicated expressions.
-

3. Java 25 Approach

Flexible Constructor Bodies changes the structure to:



This gives developers more natural constructor logic.

4. Flowchart

JEP 513



Flexible Constructor Bodies

Breaking Free from Constructor Constraints

```
// Before JEP 513
public Class() {
    super();
    validate();
}
```

→

```
// With JEP 513
public Class() {
    validate(); // Prologue
    super();
    init(); // Epilogue
}
```

Before Java 25 ❌ (won't compile)

```
java Copy

class User {
    User(String name) {
        if (name == null) {
            throw new IllegalArgumentException(
                "Name required");
        }
        super();
    }
}
```

Java 25 ✅ (valid)

```
java Copy

class User {
    User(String name) {
        if (name == null) {
            throw new IllegalArgumentException(
                "Name required");
        }
        super();
    }
}
```

Java ≤ 24

```
class Person {
    final int age;

    Person(int age) {
        this.age = age;
    }
}

class Employee extends Person {
    final String name;

    Employee(String name, int age) {
        super(age);
        if (age < 18 || age > 67) {
            throw new
IllegalArgumentException("Age must be
between 18 and 67");
        }
        this.name = name;
    }
}
```

- ❌ `super()` must be the first statement
- ❌ Move logic to static helper methods
- ❌ Harder to read and reason about

Java 25

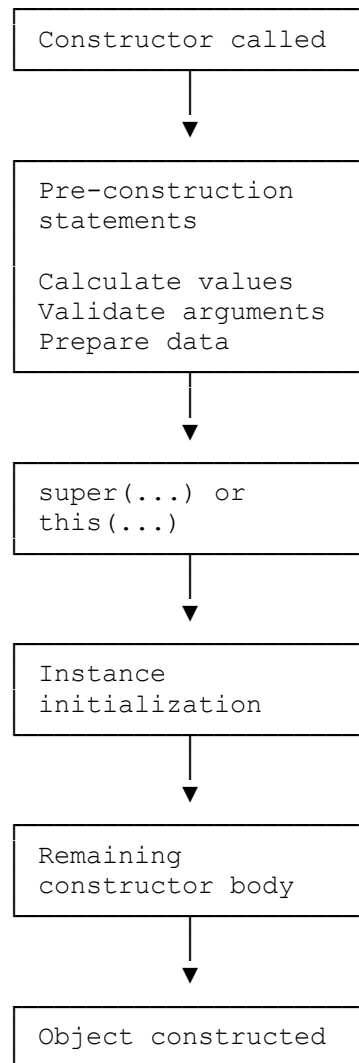
```
class Person {
    final int age;

    Person(int age) {
        this.age = age;
    }
}

class Employee extends Person {
    final String name;

    Employee(String name, int age) {
        if (age < 18 || age > 67) {
            throw new
IllegalArgumentException("Age must be
between 18 and 67");
        }
        super(age);
        this.name = name;
    }
}
```

- ✅ Logic before `super()`
- ✅ Validate early, then construct
- ✅ Cleaner and more readable



5. Java Code Example

Here is a simple Java 25 example:

```
class Parent {  
    Parent(int value) {  
        System.out.println(  
            "Parent constructor: " + value  
        );  
    }  
}  
  
public class FlexibleConstructorBodies  
    extends Parent {
```

```

FlexibleConstructorBodies(int value) {

    // Statement before super()
    int adjustedValue = value * 2;

    super(adjustedValue);

    // Normal constructor body
    System.out.println(
        "Child constructor: "
        + adjustedValue
    );
}

public static void main(String[] args) {

    new FlexibleConstructorBodies(10);
}

```

Output

Parent constructor: 20
Child constructor: 20

The important part is:

```

int adjustedValue = value * 2;

super(adjustedValue);

```

The calculation happens **before** super(...).

6. Constructor Execution Flow

For:

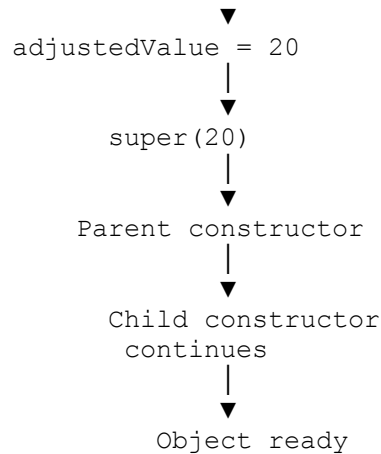
```
new FlexibleConstructorBodies(10);
```

the flow is:

```

new FlexibleConstructorBodies(10)
    |
    ▼
  Child constructor
    |
    ▼
value = 10
    |

```



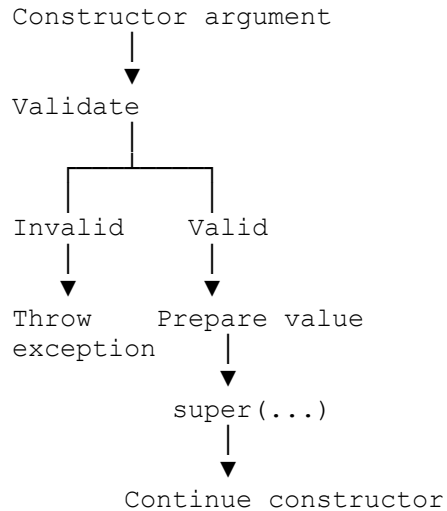
7. Validation Before super()

One useful application is validating constructor arguments before invoking the superclass constructor.

```
class Person {  
    Person(String name) {  
        System.out.println(  
            "Person created: " + name  
        );  
    }  
}  
  
public class FlexibleConstructorBodies  
    extends Person {  
    FlexibleConstructorBodies(String name) {  
        if (name == null || name.isBlank()) {  
            throw new IllegalArgumentException(  
                "Name cannot be empty"  
            );  
        }  
        super(name.trim());  
        System.out.println(  
            "Child initialization complete"  
        );  
    }  
    public static void main(String[] args) {  
        new FlexibleConstructorBodies(" Anurag ");  
    }  
}
```

```
}
```

Conceptually:



8. this(...) Invocation

Flexible constructor bodies also apply to explicit constructor invocation using `this(...)`.

For example:

```
public class FlexibleConstructorBodies {  
    FlexibleConstructorBodies(int value) {  
        int adjusted = value * 10;  
        this(adjusted, "Java");  
        System.out.println(  
            "First constructor complete"  
        );  
    }  
  
    FlexibleConstructorBodies(  
        int value,  
        String source) {  
        System.out.println(  
            "Value = " + value  
            + ", Source = " + source  
        );  
    }  
}
```

```

    };
}

public static void main(String[] args) {
    new FlexibleConstructorBodies(5);
}
}

```

The conceptual flow is:

```

Constructor A
  ↓
Calculate adjusted value
  ↓
this(adjusted, "Java")
  ↓
Constructor B
  ↓
Return to Constructor A

```

9. What Can Be Done Before super()?

The important principle is that statements before the explicit constructor invocation are allowed only when they are **safe to execute before the superclass constructor has initialized the object**.

For example, local calculations are appropriate:

```

int x = 10;
int y = x * 2;
super(y);

```

Validation of constructor arguments can also be appropriate:

```

if (value < 0) {
    throw new IllegalArgumentException();
}

```

```

super(value);

```

But you cannot treat the object as fully initialized before super().

Conceptually:

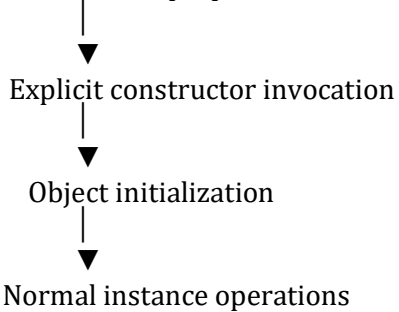
```
Before super()
  |
  ├── Local calculations ✓
  ├── Argument validation ✓
  ├── Prepare values ✓
  |
  └── Access/use uninitialized this ✗
```

10. Why Is This Safe?

Java's object-construction model requires the superclass portion of an object to be initialized before the subclass can freely use the object.

The feature therefore separates:

Statements that prepare constructor arguments



This gives flexibility **without removing the safety guarantees of constructor initialization.**

11. Before vs Java 25

Before Java 25

```
class Child extends Parent {
    Child(int x) {
        super(calculate(x));

        // body
    }

    static int calculate(int x) {
        return x * 2;
    }
}
```

```
}
```

You often needed a helper method.

Java 25

```
class Child extends Parent {  
  
    Child(int x) {  
  
        int adjusted = x * 2;  
  
        super(adjusted);  
  
        // body  
    }  
}
```

The code can be more straightforward and readable.

12. Traditional Constructor Model vs Flexible Constructor Model

Traditional Java	Java 25
super() / this() must be first statement	Statements may precede it
Limited preparation before superclass initialization	More flexible preparation
Often requires helper methods	Local calculations can be direct
Constructor invocation first	Pre-construction statements → invocation
More restrictive syntax	More expressive syntax

13. Important Relationship with final Fields

Flexible constructor bodies do **not** mean that instance fields can be freely accessed before super().

For example, this is conceptually unsafe:

```
class Child extends Parent {  
  
    int value = 10;
```

```
Child() {  
    // Cannot freely use the object here  
    super();  
}
```

The purpose of the feature is primarily to allow **pre-super preparation**, not to make the object usable before its superclass is initialized.

14. Java 25 Catalog Entry

For your Java feature catalog:

Java Version	: 25
Year	: 2025
Feature	: Flexible Constructor Bodies
UniqueClassName	: FlexibleConstructorBodies
JEP	: 513
Status	: Final

Key Takeaway

Flexible Constructor Bodies in Java 25 allow statements to execute before an explicit `super(...)` or `this(...)` constructor invocation, as long as those statements do not improperly access the object before superclass initialization. This makes it easier to validate arguments and calculate or prepare constructor arguments without using separate helper methods.

Java 25 — Ahead-of-Time Command-Line Ergonomics

Java Version: 25
Year: 2025
Feature: Ahead-of-Time Command-Line Ergonomics
UniqueClassName: AOTCommandLineErgonomics
JEP: 514
Status: Final

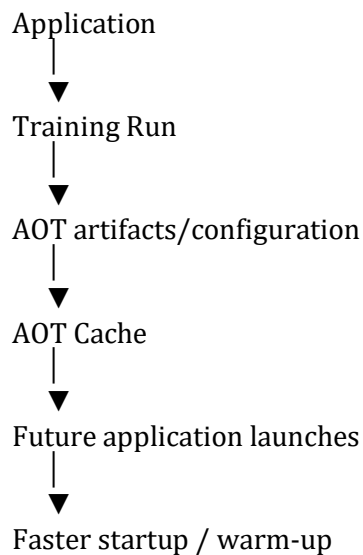
JEP 514 is part of **Project Leyden** and makes it easier to create and use **Ahead-of-Time (AOT) caches** by simplifying the command-line workflow. The goal is to improve Java application **startup time and warm-up performance** by reusing artifacts prepared during an earlier run. ([OpenJDK](#))

1. Definition

Ahead-of-Time Command-Line Ergonomics improves the java command-line interface for creating and using **AOT caches**.

Before this feature, creating an AOT cache could require multiple commands and intermediate configuration files.

Java 25 introduces a simpler workflow in which a training run can directly produce an AOT cache:



The important point is that **AOT caching is not a new Java language feature**. It is a **JVM/runtime and command-line improvement**.

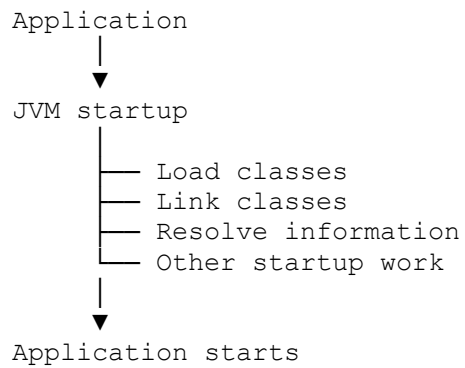
2. What Is an AOT Cache?

An **AOT cache** stores information produced before a normal production run so that the JVM does not have to perform all of the same work again during startup.

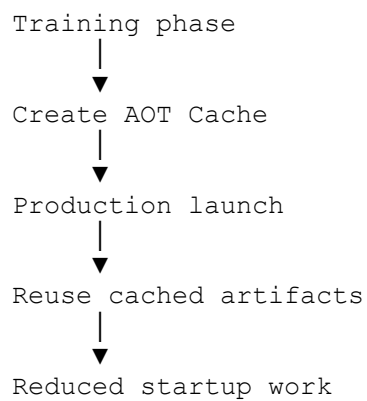
In Java 24, JEP 483 introduced the AOT cache infrastructure. Java 25's JEP 514 makes creating those caches easier. ([OpenJDK](#))

Conceptually:

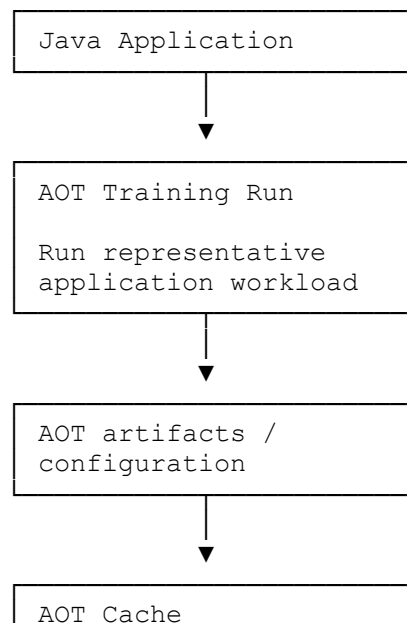
Without AOT Cache

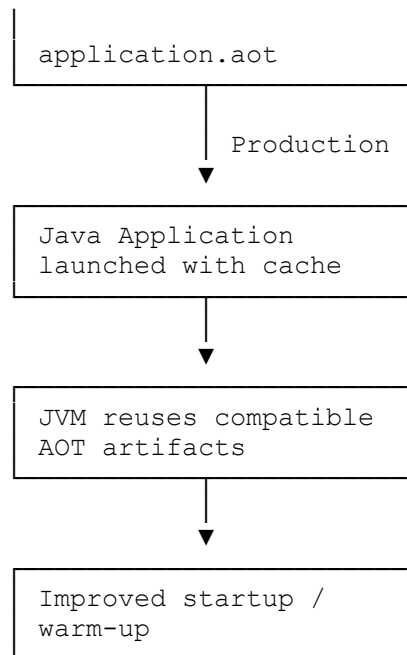


With an AOT cache:



3. Flowchart





4. Java Application Example

There is **no special Java API** that you call from application code for JEP 514.

For example, your application can simply be:

```
public class AOTCommandLineErgonomics {  
  
    public static void main(String[] args) {  
  
        System.out.println(  
            "Java application started!"  
        );  
  
        for (int i = 1; i <= 5; i++) {  
            System.out.println(  
                "Processing item " + i  
            );  
        }  
    }  
}
```

Compile it:

```
javac AOTCommandLineErgonomics.java
```

Run normally:

java AOTCommandLineErgonomics

The Java source code itself does not need to know whether an AOT cache is being used.

5. Creating an AOT Cache

The important Java 25 improvement is the simplified **one-command workflow**.

For example:

```
java -cp app.jar \  
    -XX:AOTMode=record \  
    -XX:AOTCacheOutput=app.aot \  
    AOTCommandLineErgonomics
```

The important options are:

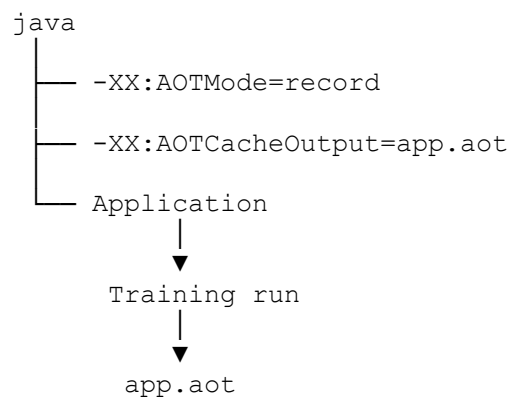
-XX:AOTMode=record

and:

-XX:AOTCacheOutput=app.aot

This allows the training run to produce the AOT cache directly. OpenJDK's implementation examples demonstrate this one-command workflow. ([OpenJDK Mail](#))

Conceptually:



6. Using the AOT Cache

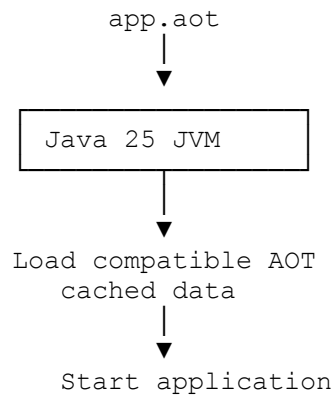
After creating:

app.aot

a later launch can use:

```
java -XX:AOTCache=app.aot \  
-cp app.jar \  
AOTCommandLineErgonomics
```

The flow becomes:

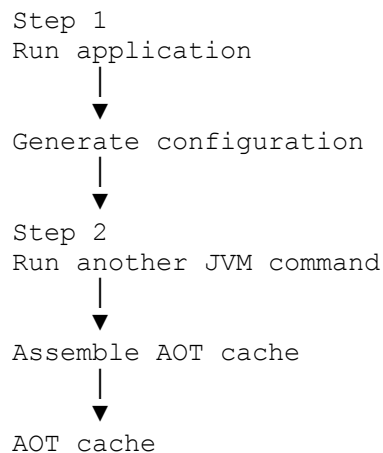


The AOT cache is tied to the application, JDK release, operating system, and CPU architecture; if those relevant characteristics change, the cache may need to be recreated. ([OpenJDK](#))

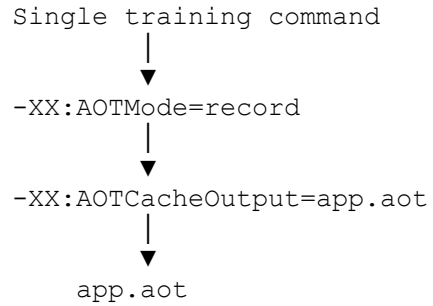
7. One-Command Workflow

This is the key ergonomic improvement.

Earlier conceptual workflow



Java 25 JEP 514 workflow



OpenJDK specifically describes this as a "**one-command workflow**" for creating an AOT cache. ([OpenJDK Mail](#))

8. Complete Example

Suppose you have:

```
project/
├── AOTCommandLineErgonomics.java
└── app.jar
```

Step 1 — Normal execution

```
java -cp app.jar AOTCommandLineErgonomics
```

Step 2 — Training and AOT-cache creation

```
java -cp app.jar \
  -XX:AOTMode=record \
  -XX:AOTCacheOutput=app.aot \
  AOTCommandLineErgonomics
```

Result:

app.aot

Step 3 — Production execution using the cache

```
java -XX:AOTCache=app.aot \
  -cp app.jar \
  AOTCommandLineErgonomics
```

9. %p and %t File Naming

JEP 514 also supports substitutions useful when generating cache/configuration files.

For example:

```
java \
  -XX:AOTConfiguration=config%p.aotconfig \
  -XX:AOTCacheOutput=cache%p.aot \
  -cp app.jar \
  AOTCommandLineErgonomics
```

%p can represent the JVM process ID in the relevant AOT file name. %t can represent the JVM startup timestamp. ([OpenJDK](#))

Conceptually:

```
cache%p.aot
  |
  ▼
cache12345.aot
```

This is useful when multiple training processes need separate output files.

10. JAVA_AOT_OPTIONS

The AOT workflow also provides an environment-variable mechanism for supplying additional JVM options during AOT cache creation.

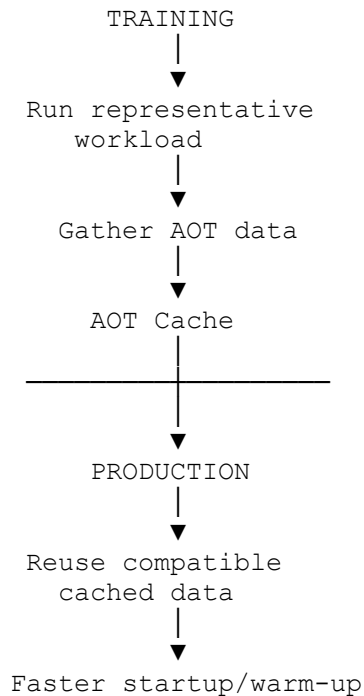
Conceptually:

```
Environment
  |
  ▼
JAVA_AOT_OPTIONS
  |
  ▼
AOT cache creation process
```

OpenJDK's implementation notes describe support for this mechanism as part of JEP 514. ([OpenJDK Mail](#))

11. Training vs Production

A useful mental model is:



The training workload should be representative of what the application normally does.

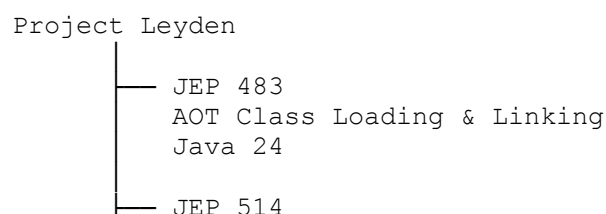
12. Relationship with Project Leyden

JEP 514 is part of **Project Leyden**, whose broader goal is to improve Java application:

- startup time,
- time to peak performance,
- footprint.

Project Leyden has delivered several AOT-related JEPs, including JEP 483 in JDK 24 and JEPs 514 and 515 in JDK 25. ([OpenJDK](#))

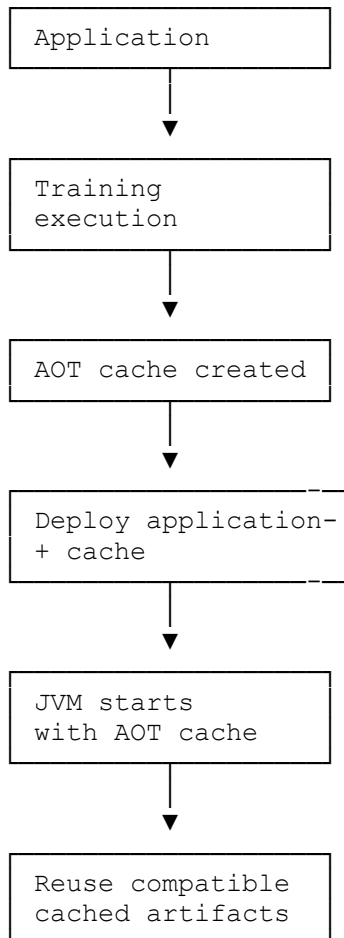
The progression can be viewed as:



AOT Command-Line Ergonomics
Java 25

JEP 515
AOT Method Profiling
Java 25

13. AOT Cache Lifecycle

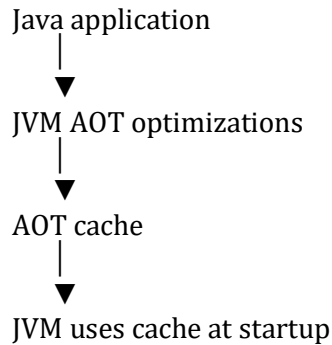


14. Important Distinction

AOT caching does **not** mean:

Java source
↓
Native executable

It is better understood as:



The application continues to run on the **JVM**.

15. Java 25 Catalog Entry

For your Java feature catalog:

Java Version	: 25
Year	: 2025
Feature	: Ahead-of-Time Command-Line Ergonomics
UniqueClassName	: AOTCommandLineErgonomics
JEP	: 514
Status	: Final

JEP 514 is officially listed as a **JDK 25 final feature**. ([OpenJDK](#))

Key Takeaway

Ahead-of-Time Command-Line Ergonomics (JEP 514) in Java 25 simplifies the process of creating and using AOT caches. Instead of managing multiple configuration and assembly commands, developers can use a streamlined training command such as - XX:AOTMode=record -XX:AOTCacheOutput=app.aot, then launch the application with - XX:AOTCache=app.aot to reuse compatible AOT artifacts and improve startup and warm-up performance.

Java 25 — Ahead-of-Time Method Profiling

Java Version: 25

Year: 2025

Feature: Ahead-of-Time Method Profiling

UniqueClassName: AOTMethodProfiling

JEP: 515

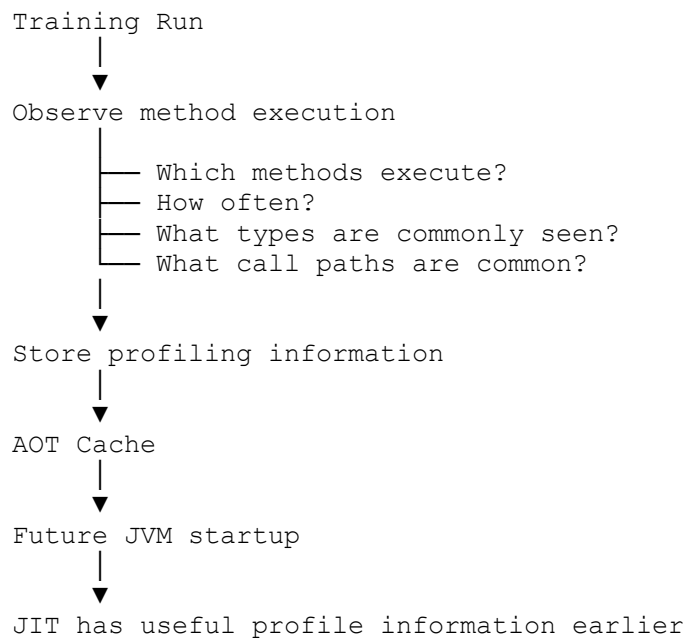
Status: Final

1. Definition

Ahead-of-Time Method Profiling is a Java 25 performance feature from **Project Leyden** that allows the JVM to record **method execution profiles during an earlier training run** and preserve that information in an **AOT cache**.

The JVM's JIT compiler normally needs to observe an application while it runs before it can make some of its best optimization decisions.

The basic idea of AOT Method Profiling is:



This can reduce the amount of time required for the JVM to reach highly optimized performance.

2. Why Method Profiling Matters

The Java JIT compiler performs optimizations based partly on information it collects while an application executes.

For example:

```
result = calculate(value);
```

The JVM may initially know little about `calculate()`.

After observing many executions, it can learn things such as:

```
calculate()
├── called 1,000,000 times
├── usually receives Integer
└── usually follows the same execution path
```

That information can help the JIT make better optimization decisions.

Without AOT method profiling:

Application starts



Run application



Collect profiles



JIT optimizations



Peak performance

With AOT method profiling:

Training run



Collect method profiles



Store in AOT cache



Future application startup

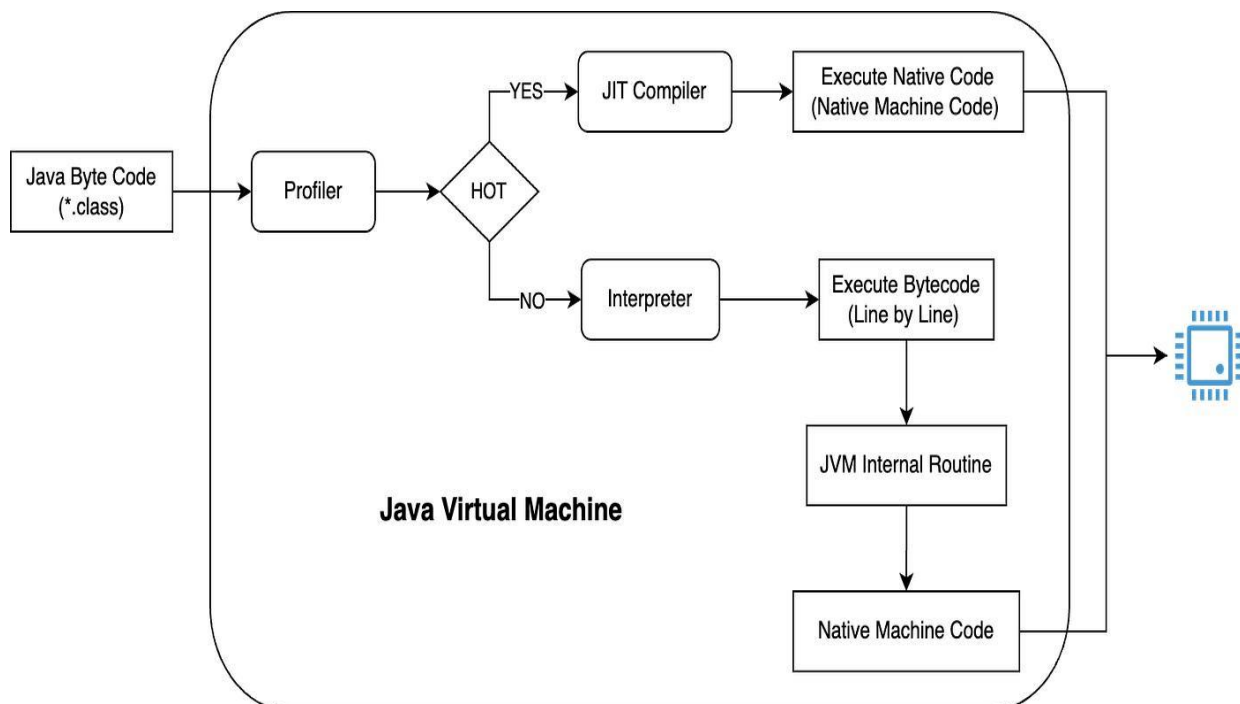
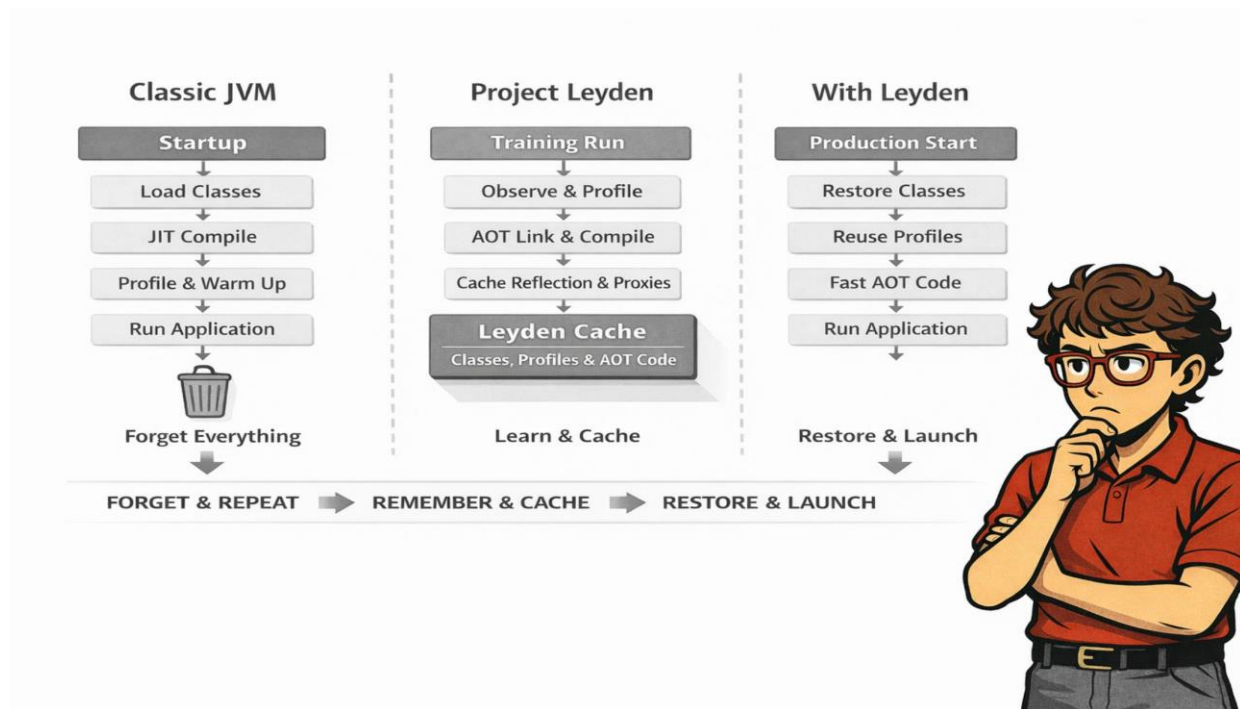


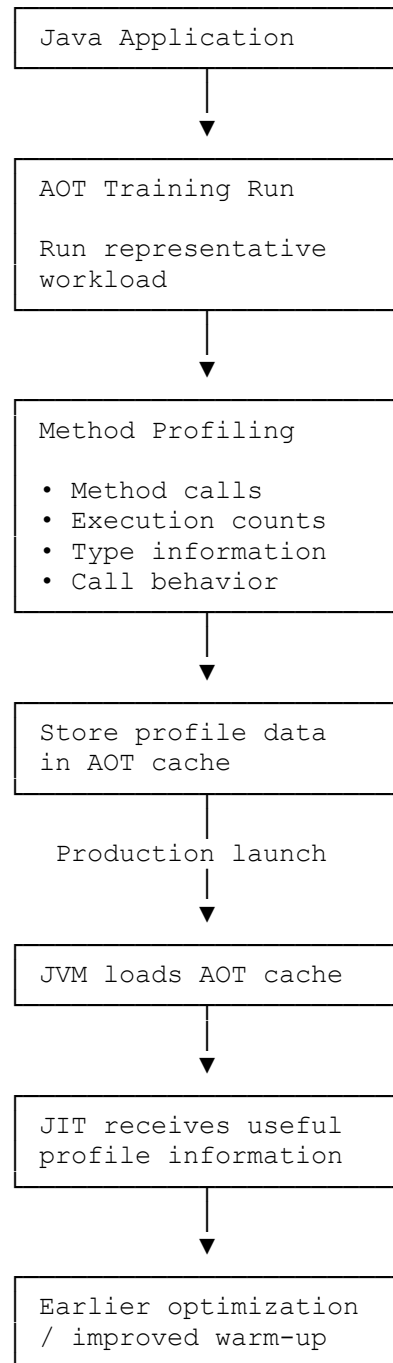
Reuse profile information



Earlier optimization

3. Flowchart





4. Java Code Example

Like **AOT Command-Line Ergonomics**, this feature does **not introduce a new Java language construct** that you call from source code.

Your Java application remains ordinary Java:

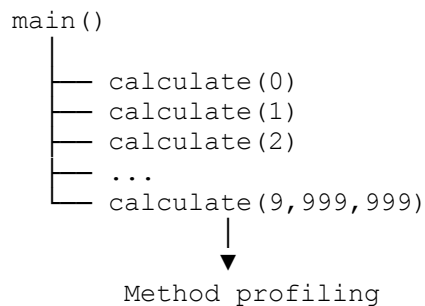
```
public class AOTMethodProfiling {  
  
    public static void main(String[] args) {  
  
        System.out.println(  
            "Starting application..."  
        );  
  
        long result = 0;  
  
        for (int i = 0; i < 10_000_000; i++) {  
            result += calculate(i);  
        }  
  
        System.out.println(  
            "Result = " + result  
        );  
    }  
  
    static long calculate(int value) {  
  
        return value * 2L + 10;  
    }  
}
```

The important method for profiling is:

```
static long calculate(int value)
```

During a training run, the JVM can observe that `calculate()` is executed repeatedly.

Conceptually:



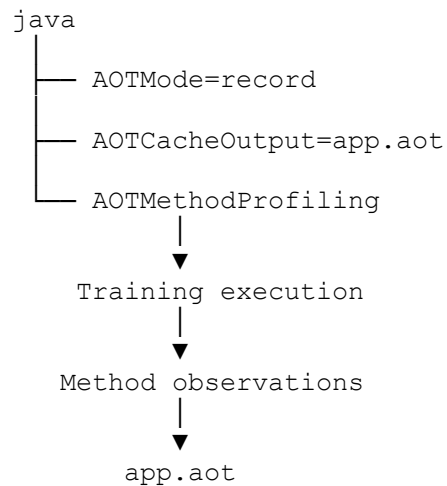
5. Creating an AOT Profile

The AOT workflow can be used to record method profiles.

A representative command is:

```
java -XX:AOTMode=record \  
      -XX:AOTCacheOutput=app.aot \  
      -cp app.jar \  
      AOTMethodProfiling
```

Conceptually:



The exact AOT command-line workflow is part of the Java 25 AOT improvements delivered through JEP 514.

6. Using the AOT Cache

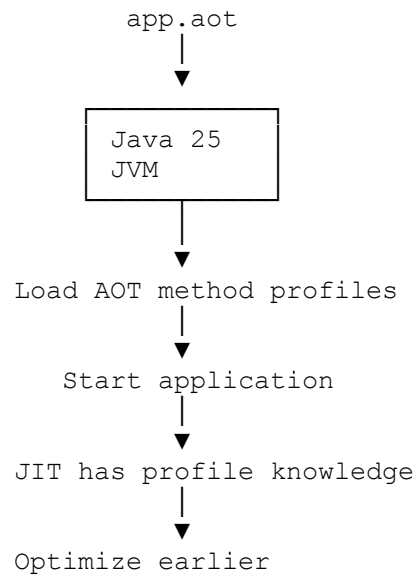
After the training run produces:

app.aot

the application can subsequently be launched using:

```
java -XX:AOTCache=app.aot \  
      -cp app.jar \  
      AOTMethodProfiling
```

Conceptually:



7. Example of Profile Information

Suppose the training workload executes:

```
calculate(10);  
calculate(20);  
calculate(30);
```

millions of times.

The JVM can build profile information conceptually similar to:

Method: `calculate(int)`

Invocation count:
10,000,000

Argument type:
int

Common execution:
multiplication
addition
return

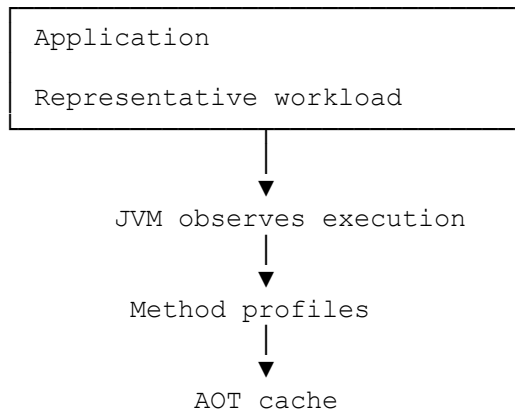
That information can be made available through the AOT mechanism.

The actual profile representation is an internal JVM implementation detail—not a Java object that application code manipulates directly.

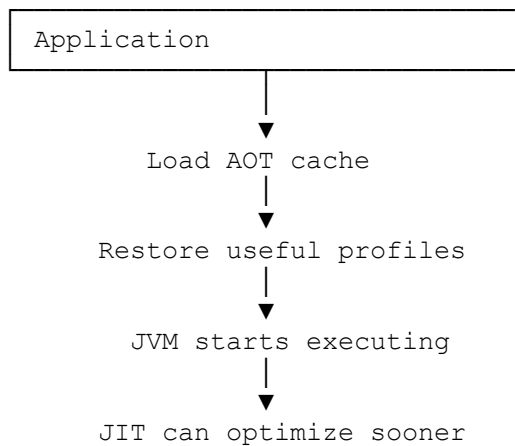
8. Training vs Production

The most important concept is the distinction between the **training run** and the **production run**.

Training



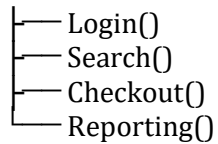
Production



9. Why Representative Workload Matters

Suppose your application has:

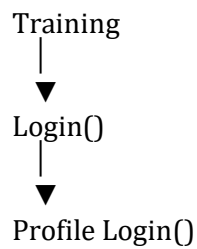
Application
|



But your training run only executes:

Login()

then the AOT profile will primarily contain information about the login path.



It won't magically know how your application behaves under:

Search()
Checkout()
Reporting()

Therefore, the training workload should ideally exercise the application's important execution paths.

10. AOT Method Profiling vs AOT Class Loading

These are related but different.

AOT Class Loading & Linking

Primarily addresses:

Loading
+
Linking
+
Class-related startup work

AOT Method Profiling

Primarily addresses:

Runtime execution profiles

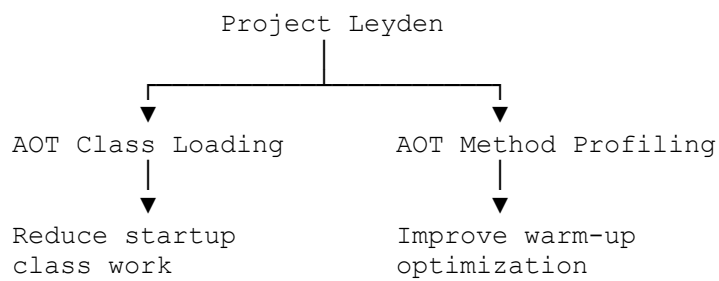


JIT optimization decisions



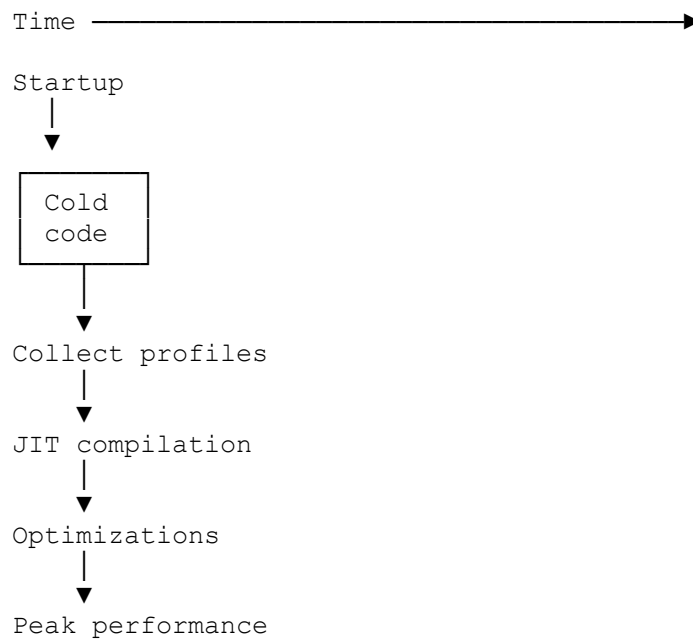
Faster warm-up

Conceptually:



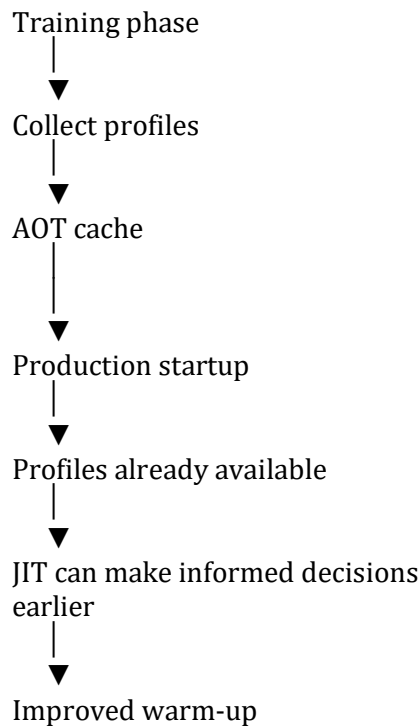
11. Traditional JVM Warm-Up

Without AOT method profiling:



The application needs to **run long enough** for the JVM to collect useful information.

12. With AOT Method Profiling

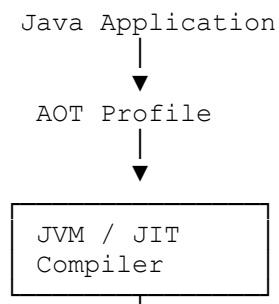


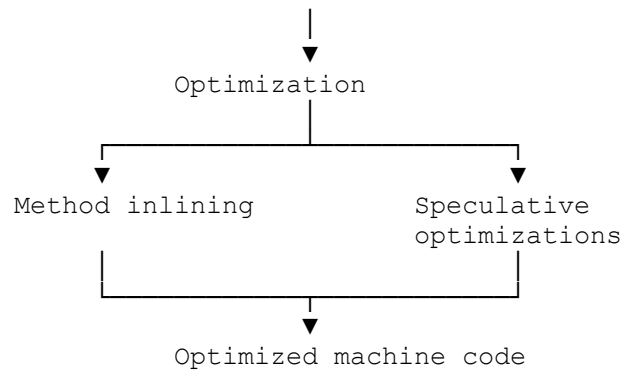
This is especially interesting for applications where:

- startup matters,
- instances are short-lived,
- applications frequently restart,
- serverless/container workloads need quick readiness,
- JVM warm-up represents a significant fraction of total execution time.

13. AOT Method Profiling and JIT

The relationship can be summarized as:





AOT method profiling therefore complements rather than replaces the JIT compiler.

14. Java 25 Catalog Entry

For your Java feature catalog:

Java Version	: 25
Year	: 2025
Feature	: Ahead-of-Time Method Profiling
UniqueClassName	: AOTMethodProfiling
JEP	: 515
Status	: Final

The feature is part of **Project Leyden's AOT work** and was delivered in JDK 25.

Key Takeaway

Ahead-of-Time Method Profiling (JEP 515) allows Java 25 to preserve useful method execution profiles in an AOT cache during a training run. On subsequent launches, the JVM can reuse that profile information to make JIT compilation and optimization decisions earlier, helping reduce application warm-up time while retaining the JVM's dynamic optimization capabilities.

Java 25 — JFR Cooperative Sampling

Java Version: 25

Year: 2025

Feature: JFR Cooperative Sampling

UniqueClassName: JFRCooperativeSampling

JEP: 518

Status: Final

JEP 518 improves the stability and accuracy of **Java Flight Recorder (JFR)** when it asynchronously samples Java thread stacks. It changes the sampling approach so that stack walking occurs at **safe points**, while using cooperation between the sampler and the sampled thread to reduce the usual **safe point bias**. ([OpenJDK Mail](#))

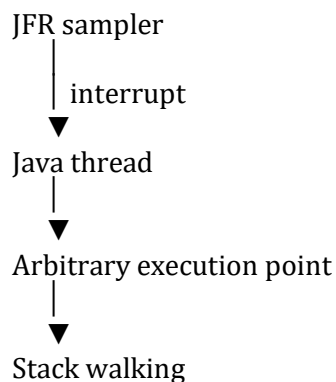
1. Definition

JFR Cooperative Sampling is a JVM/JFR implementation improvement that changes how JFR obtains asynchronous execution samples.

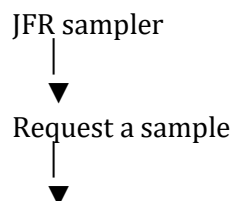
Traditional sampling can interrupt a Java thread at an arbitrary point and then attempt to walk its stack while that thread is suspended. Walking a stack at arbitrary execution points is complicated and can introduce stability problems.

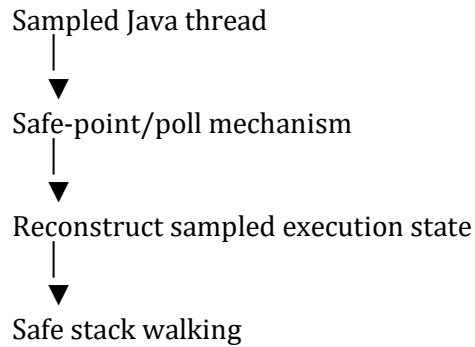
Cooperative sampling instead works approximately like this:

Traditional sampling



With cooperative sampling:





The goal is to obtain safer stack traces **without simply sampling only when a thread naturally reaches a safepoint**, which would create sampling bias. (bugs.openjdk.org)

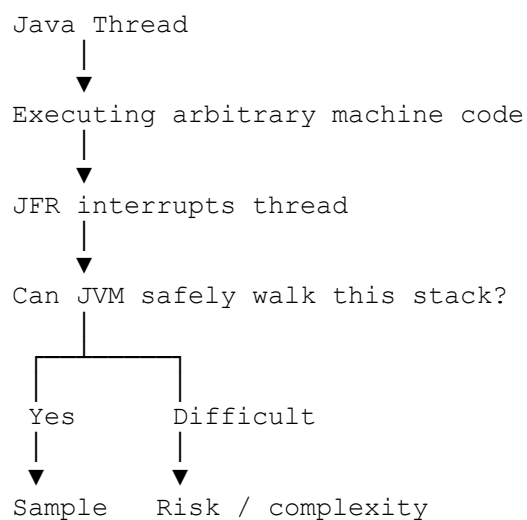
2. What Problem Does It Solve?

JFR needs stack samples to answer questions such as:

- Which methods consume CPU?
- What code is executing?
- Where is an application spending its time?
- Which call paths are hot?
- What are threads doing during a recording?

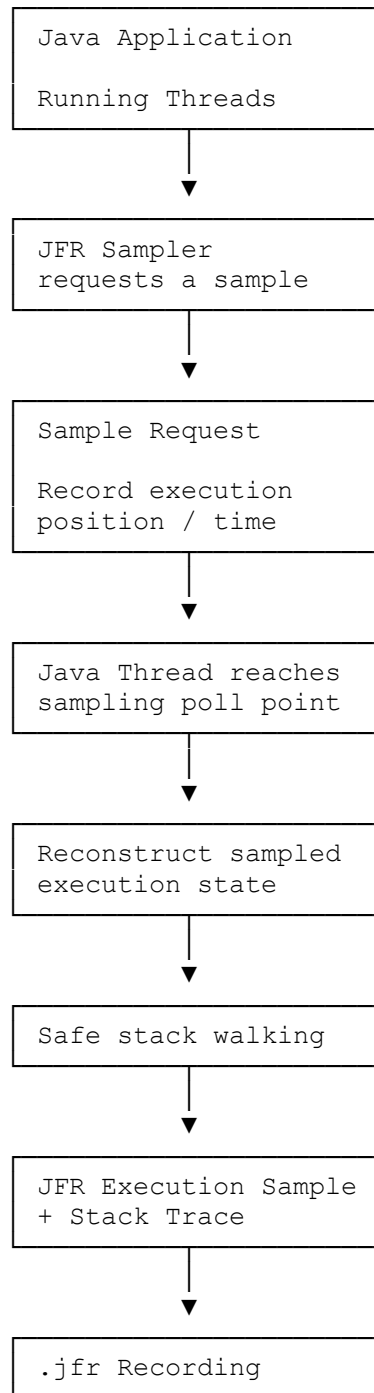
Traditional asynchronous stack walking can be technically difficult because the thread might be executing at a point where its stack is not straightforward to traverse safely.

The problem can be visualized as:



JEP 518 changes this model so stack walking happens at locations where the JVM has sufficient information to safely reconstruct the sampled state. (bugs.openjdk.org)

3. Flowchart



The implementation uses a **JFR sample request** containing execution-state information such as the program counter, stack pointer, bytecode pointer where applicable, and sampling timestamp. (bugs.openjdk.org)

4. Java Code Example

JFR Cooperative Sampling is primarily a **JVM implementation feature**, so there is **no new Java class or method that application code calls directly**.

You can, however, create an application whose execution can be sampled by JFR.

```
public class JFRCooperativeSampling {  
    public static void main(String[] args) {  
        System.out.println(  
            "Starting CPU-intensive application..."  
        );  
        long result = 0;  
        for (long i = 0; i < 5_000_000_000L; i++) {  
            result += calculate(i);  
        }  
        System.out.println(  
            "Result = " + result  
        );  
    }  
    static long calculate(long value) {  
        return value * 31 + 7;  
    }  
}
```

The application repeatedly executes:

```
calculate(i)
```

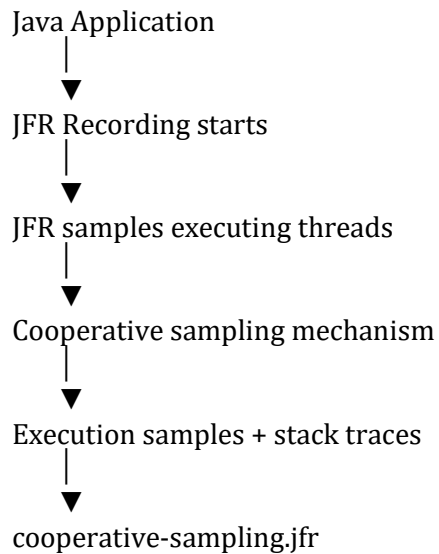
which gives JFR execution sampling opportunities.

5. Start a JFR Recording

You can run the application with JFR enabled:

```
java -XX:StartFlightRecording=\
duration=20s,\
filename=cooperative-sampling.jfr \
JFRCooperativeSampling
```

Conceptually:



JFR recordings are binary .jfr files that can subsequently be inspected with JFR tooling. ([OpenJDK](#))

6. Inspect the Recording

After execution:

cooperative-sampling.jfr

you can use the JFR command-line tool:

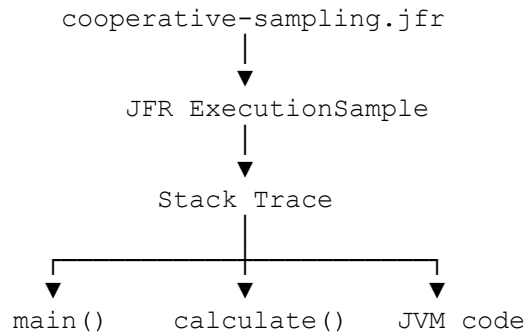
```
jfr summary cooperative-sampling.jfr
```

or:

```
jfr print --events jdk.ExecutionSample \
cooperative-sampling.jfr
```

The JFR API/tooling exposes execution-sample events and their stack traces. ([OpenJDK](#))

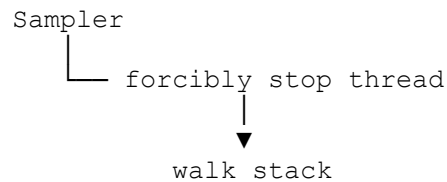
Conceptually:



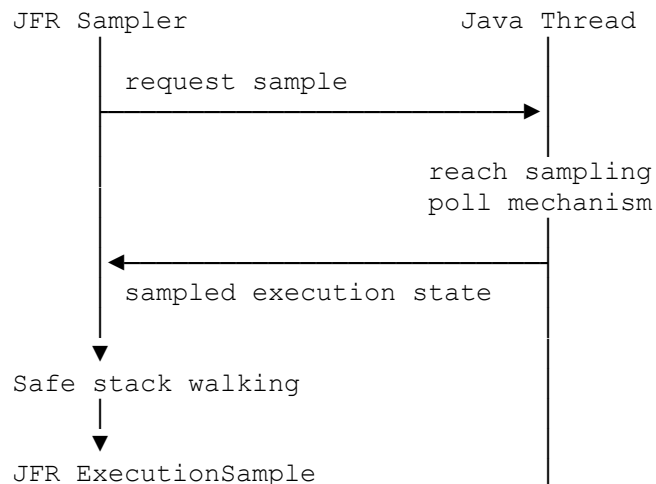
7. What Is "Cooperative"?

The word **cooperative** refers to the fact that the **sampled thread participates in producing a safely walkable representation of its execution state**.

It is not simply:



Instead, the model is closer to:



This cooperation allows the JVM to use safe locations for stack walking while preserving information about where the thread was executing when the sample was requested.

(bugs.openjdk.org)

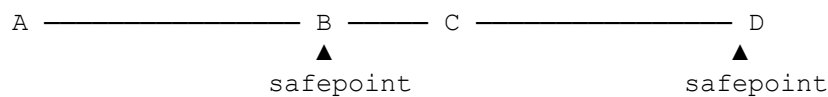
8. Safepoint Bias

One challenge is **safepoint bias**.

If a profiler only samples when a thread happens to reach a safepoint, frequently executed code near safepoints could be overrepresented while code that rarely reaches safepoints could be underrepresented.

For example:

Actual execution:



A simplistic safepoint-only sampler might repeatedly observe:

B
D
B
D
B
D

even though the thread spends substantial time executing elsewhere.

JEP 518 therefore aims to combine **safe stack walking** with a mechanism that minimizes this bias.
([OpenJDK Mail](#))

9. Traditional vs Cooperative Sampling

Traditional JFR sampling	Cooperative JFR sampling
Asynchronous sampling	Cooperative sampling
Stack may need to be walked from arbitrary execution state	Stack walking occurs at safe locations
More complicated stack-safety problem	Improved stack-walking safety
Can require crash-protection mechanisms	Designed to reduce this complexity
Potential sampling challenges	Reduces safepoint bias while maintaining safety

Older JFR sampling implementation	Java 25 JEP 518 implementation
--	--------------------------------

The primary goal is **stability**, while also improving sampling behavior and avoiding excessive safepoint bias. (bugs.openjdk.org)

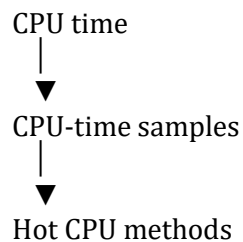
10. Relationship to JFR CPU-Time Profiling

This is especially useful to distinguish from the previous Java 25 feature you asked about.

JFR CPU-Time Profiling — JEP 509

Answers:

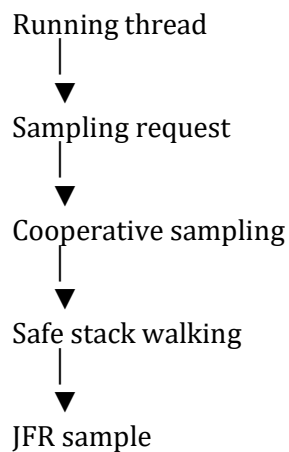
"Where is CPU time being consumed?"



JFR Cooperative Sampling — JEP 518

Answers more fundamentally:

"How can JFR safely and accurately obtain execution stack samples?"

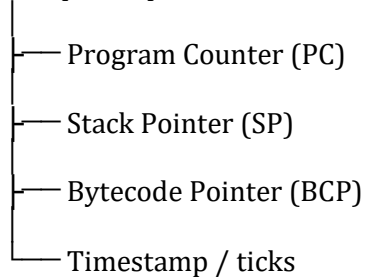


So JEP 518 is primarily a **sampling infrastructure improvement**, whereas JEP 509 introduces **CPU-time-based profiling**.

11. Internal Conceptual Model

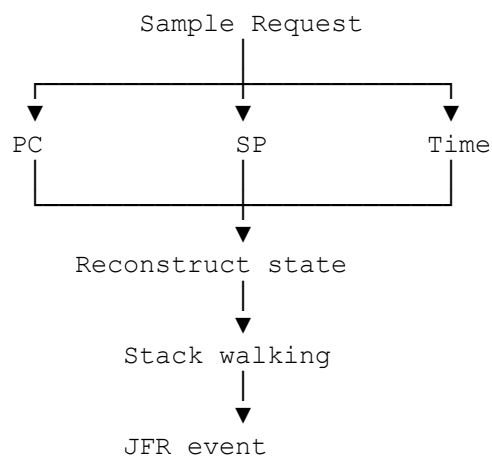
The implementation represents sampled execution state using a structure conceptually like:

JFR Sample Request



The Java thread can then reconstruct the sampled execution state when it reaches the appropriate sampling mechanism. (bugs.openjdk.org)

Simplified:



12. Why It Matters

JFR is designed for **low-overhead production profiling and diagnostics**.

Cooperative sampling improves the underlying mechanism used to obtain stack samples, which is important for:

- application profiling,
- CPU analysis,
- performance troubleshooting,
- production diagnostics,
- identifying hot methods,

- reducing sampling-related JVM instability.

The JEP specifically targets improved stability when JFR asynchronously samples Java thread stacks. ([OpenJDK Mail](#))

13. Java 25 Catalog Entry

For your Java feature catalog:

Java Version	: 25
Year	: 2025
Feature	: JFR Cooperative Sampling
UniqueClassName	: JFRCooperativeSampling
JEP	: 518
Status	: Final

JEP 518 is officially delivered in **JDK 25**. ([OpenJDK](#))

Key Takeaway

JFR Cooperative Sampling (JEP 518) improves Java Flight Recorder's stack-sampling infrastructure by having sampled threads cooperate with the sampling mechanism, allowing stack walking at safe locations while minimizing safepoint bias. The result is a safer and more robust foundation for JFR execution and profiling samples.

Java 25 — Compact Object Headers

Java Version: 25
Year: 2025
Feature: Compact Object Headers
UniqueClassName: CompactObjectHeaders
JEP: 519
Status: Final

1. Definition

Compact Object Headers is a Java 25 JVM feature that reduces the memory footprint of ordinary Java objects by making their **object headers smaller**.

Every Java object has JVM-internal metadata associated with it. This metadata can include information such as:

- the object's class/type information,
- synchronization/locking state,
- identity-related information,
- garbage-collection-related state.

Traditionally, object headers can consume a significant amount of memory, especially when an application creates **millions or billions of small objects**.

JEP 519 introduces a more compact object-header representation that can reduce object memory overhead.

The key idea is:

Traditional Object

Larger Object Header
Instance Fields

Compact Object

Smaller Object Header
Instance Fields

2. What Is an Object Header?

An object header is **JVM implementation metadata** stored with an object.

For example, conceptually:

Java Object

Object Header
Class information
Synchronization information
GC-related information
Object fields
int id
reference name

The exact layout is JVM implementation-specific.

It is important to understand that:

Object headers are not Java fields and cannot normally be accessed directly from Java source code.

3. Why Compact Object Headers?

Consider an application containing:

1,000,000,000 small objects

Even a small reduction in per-object overhead can produce substantial memory savings.

Conceptually:

Large Header



Every object carries more metadata



Higher heap usage



More memory pressure

Compact Header



Less metadata per object



Lower heap usage



Potentially better cache behavior

This is especially valuable for object-heavy applications.

4. Flowchart

Compact Object Header (64 bits)

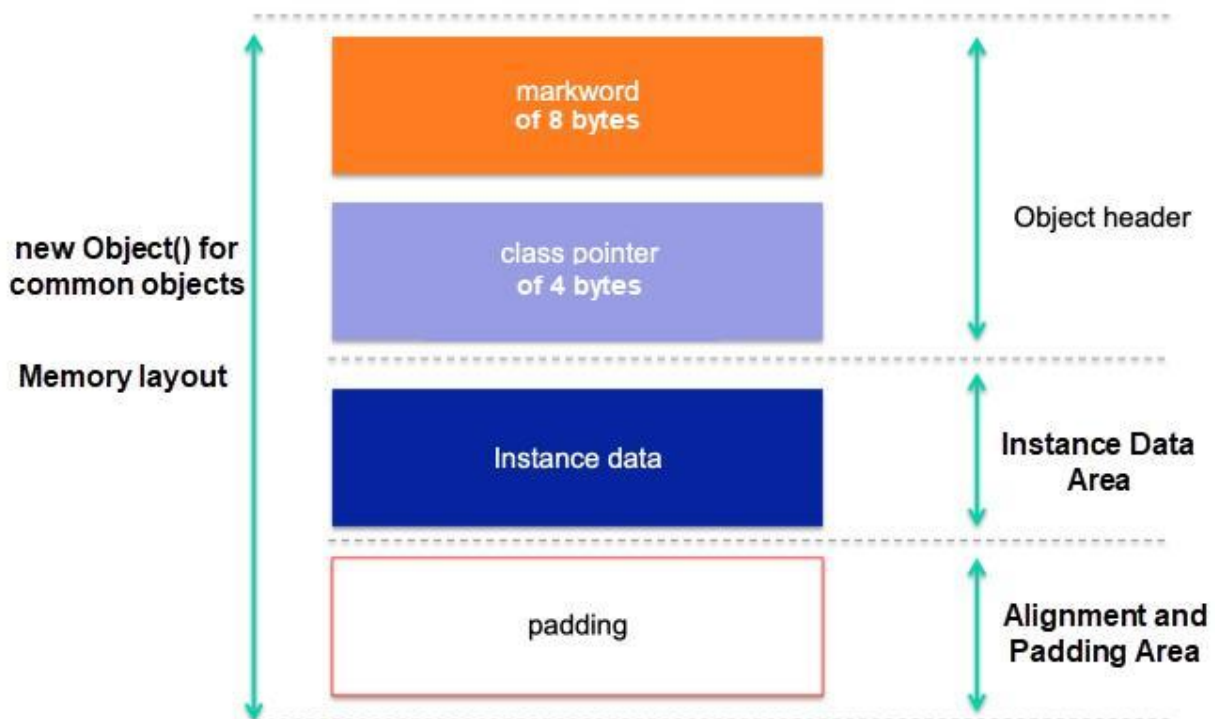
Header (compact):



[CCCCCCCCCCCCCCCCCCCCCCHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHVVVVVAAAATT]

|----- 22 bits -----| |----- 31 bits -----| |4b| |4b| s | tt

- Total size: 64 bits (8 bytes) — 33% smaller than traditional headers
- Supports ~4 million classes • Preserves full hash code range • No header overwrite for locks

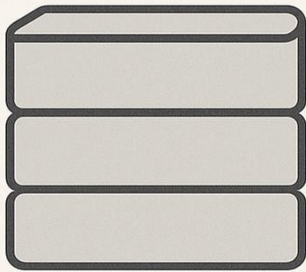




Compact Object Headers

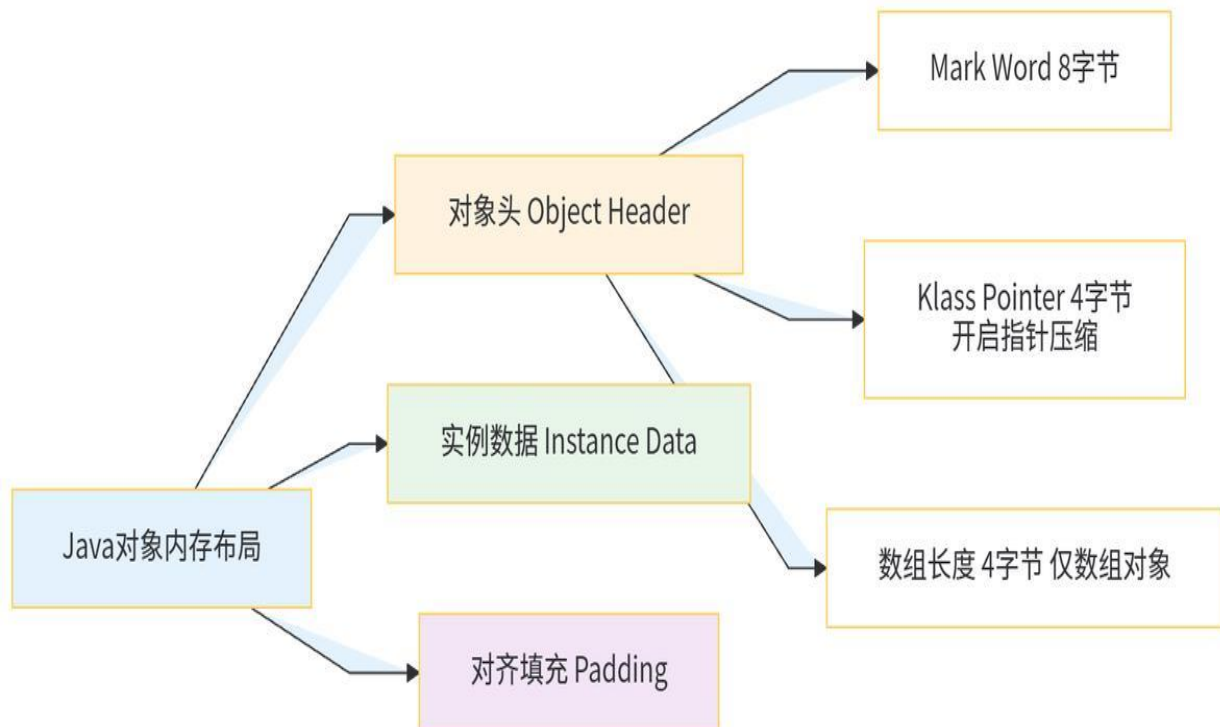
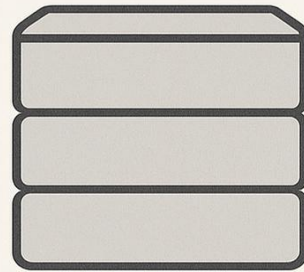
Before

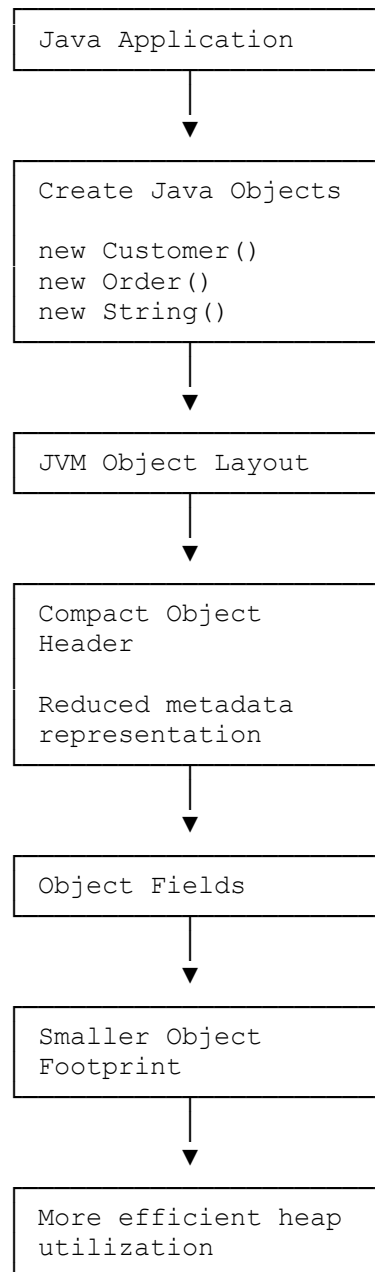
12 bytes per object



After

8 bytes per object





5. Java Code Example

Unlike a normal Java language feature, **Compact Object Headers** does not introduce a new Java class, method, or syntax.

Your Java program remains ordinary Java code.

For example:

```

public class CompactObjectHeaders {

    static class Customer {

        private final int id;
        private final String name;

        Customer(int id, String name) {
            this.id = id;
            this.name = name;
        }
    }

    public static void main(String[] args) {

        Customer[] customers =
            new Customer[1_000_000];

        for (int i = 0; i < customers.length; i++) {

            customers[i] =
                new Customer(
                    i,
                    "Customer-" + i
                );
        }

        System.out.println(
            "Created "
                + customers.length
                + " Customer objects."
        );
    }
}

```

Output:

Created 1000000 Customer objects.

The Java code does not explicitly request a compact header.

Instead, the **JVM controls the object layout**.

6. JVM-Level Configuration

Compact Object Headers is controlled through JVM configuration rather than Java source syntax.

For example, when supported by the JDK build, you can inspect the relevant VM option:

```
java -XX:+PrintFlagsFinal -version | findstr CompactObjectHeaders
```

On Linux/macOS:

```
java -XX:+PrintFlagsFinal -version | grep CompactObjectHeaders
```

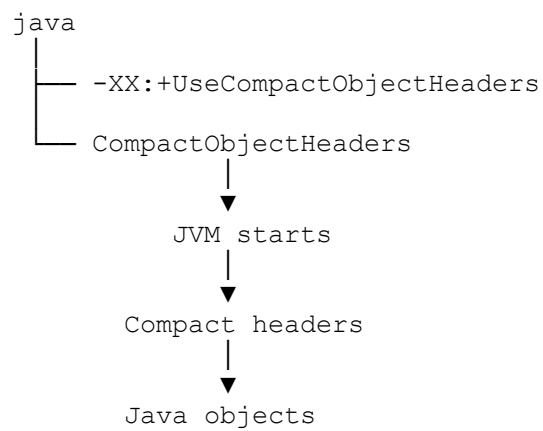
The feature is associated with the JVM option:

```
-XX:+UseCompactObjectHeaders
```

A Java 25 application can therefore be launched with:

```
java -XX:+UseCompactObjectHeaders CompactObjectHeaders
```

Conceptually:



7. Traditional vs Compact Object Layout

A simplified representation:

Traditional layout

Object

Header 12/16 bytes*
Fields int id reference name

Compact header layout

Object

--

Compact Header smaller metadata
Fields int id reference name

* Exact object-header size depends on JVM configuration, architecture, compressed references/class pointers, object alignment, and other implementation details. Therefore, the diagrams are conceptual rather than universal byte layouts.

8. Why Small Objects Matter

Suppose an application creates many objects:

```
Customer customer = new Customer(...);
Order order = new Order(...);
Product product = new Product(...);
```

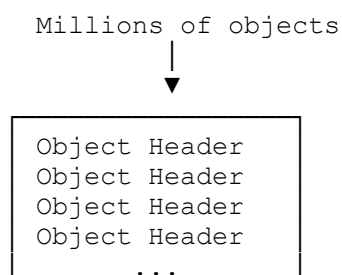
Each object has:

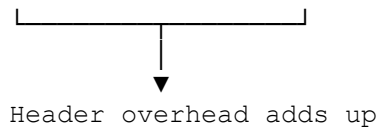
```
Header
+
Fields
+
Alignment/padding
```

For a very large number of small objects:

```
10 bytes saved/object
×
1,000,000 objects
=
10 MB
```

Conceptually:

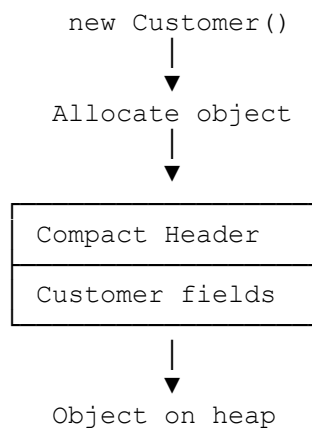




Even a modest reduction in object metadata can therefore have a meaningful aggregate effect.

9. Object Layout Flow

When this feature is enabled, conceptually:



The application still sees:

```
Customer customer = new Customer(...);
```

The JVM handles the layout internally.

10. Relationship with Compressed Class Pointers

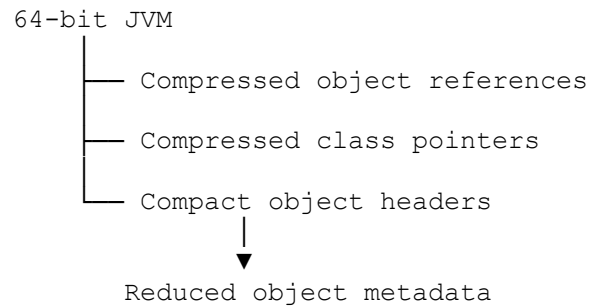
Compact Object Headers is closely related to the JVM's existing techniques for reducing object metadata.

Historically, HotSpot has used mechanisms such as:

- Compressed ordinary object pointers
- Compressed class pointers

to reduce memory overhead on 64-bit systems.

Conceptually:



JEP 519 goes further by redesigning how the header itself represents necessary information.

11. Synchronization and Object Identity

Object headers have historically been involved in JVM mechanisms such as:

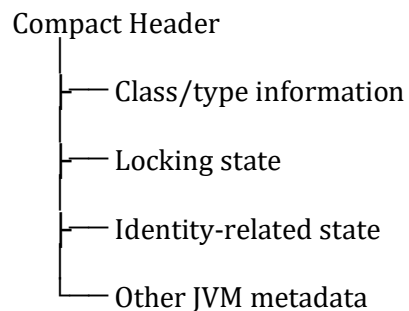
```
synchronized (object) {  
    // ...  
}
```

and object identity:

```
Object object = new Object();  
  
System.out.println(  
    System.identityHashCode(object)  
);
```

The compact-header implementation must therefore preserve the JVM's required semantics for operations involving object identity and synchronization.

Conceptually:



The JVM handles these details transparently to application code.

12. Compact Object Headers vs Java Code

This is an important distinction for your Java feature catalog.

Java language feature

Example:

```
record Customer(int id, String name) {}
```

The source language changes.

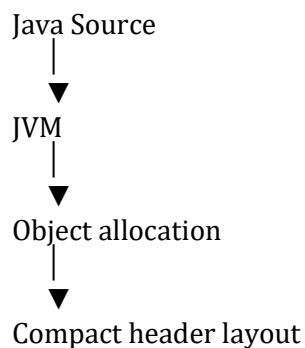
Compact Object Headers

Example:

```
Customer customer =  
    new Customer(1, "Anurag");
```

The source code **does not change**.

Instead:



Therefore, developers benefit from the feature without changing normal object-creation syntax.

13. Memory-Usage Concept

Consider:

Traditional

```
Object 1 → Header + Fields  
Object 2 → Header + Fields  
Object 3 → Header + Fields  
...
```

Object N → Header + Fields

Compact

Object 1 → Small Header + Fields
Object 2 → Small Header + Fields
Object 3 → Small Header + Fields
...
Object N → Small Header + Fields

As N becomes very large:

Memory savings
 $\approx$
Header reduction × number of objects

The actual savings depend heavily on object layout, alignment, references, JVM options, and application object sizes.

14. Java 25 Catalog Entry

For your Java feature catalog:

Java Version	: 25
Year	: 2025
Feature	: Compact Object Headers
UniqueClassName	: CompactObjectHeaders
JEP	: 519
Status	: Final

Key Takeaway

Compact Object Headers (JEP 519) in Java 25 reduces the memory overhead associated with Java object headers by using a more compact JVM object-header representation. It is a JVM implementation feature rather than a Java language/API feature, so existing Java source code continues to work normally while object-heavy applications can benefit from reduced heap usage and potentially improved memory/cache efficiency.

Java 25 — JFR Method Timing & Tracing

Java Version: 25

Year: 2025

Feature: JFR Method Timing & Tracing

UniqueClassName: JFRMethodTimingTracing

JEP: 520

Status: Final

1. Definition

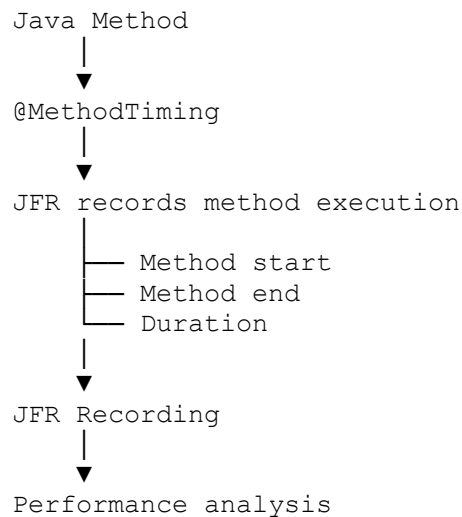
JFR Method Timing & Tracing is a Java 25 enhancement to **Java Flight Recorder (JFR)** that provides a lightweight way to record the **execution time of selected Java methods**.

It is designed for situations where you want to answer questions such as:

- How long does this method take?
- How often is this method called?
- Which methods are taking the most time?
- What is the call relationship between selected methods?

The feature introduces the `@MethodTiming` annotation in the `jdk.jfr` package.

Conceptually:



2. Why Method Timing Is Useful

Suppose an application contains:

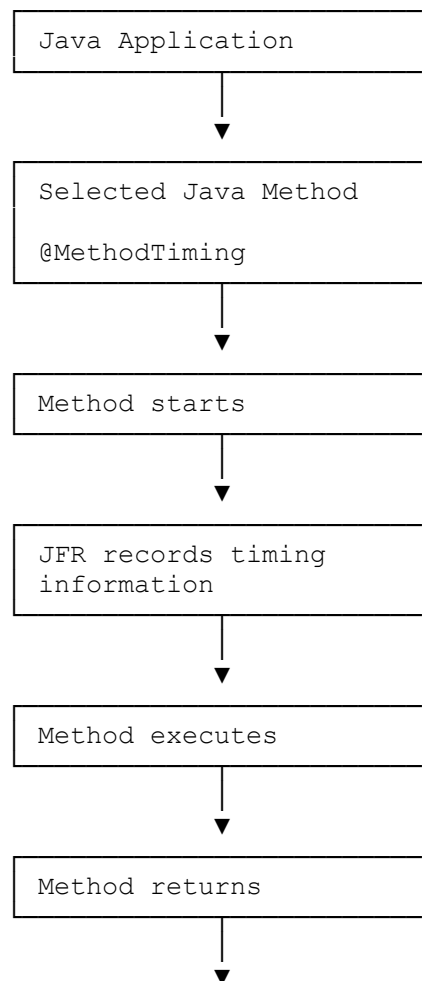
```
processOrder()  
├── validateOrder()  
├── calculatePrice()  
└── saveOrder()
```

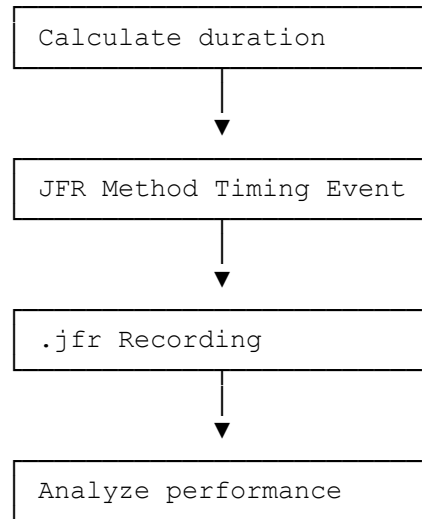
You may want to know:

```
processOrder()    → 150 ms  
validateOrder()   → 10 ms  
calculatePrice()  → 20 ms  
saveOrder()       → 120 ms
```

JFR method timing helps expose this information without requiring you to manually insert timing code throughout the application.

3. Flowchart





4. Java Code Example

The important API is:

```
import jdk.jfr.MethodTiming;
```

A method can be annotated for timing:

```
import jdk.jfr.MethodTiming;

public class JFRMethodTimingTracing {

    public static void main(String[] args) {

        processOrder();
    }

    @MethodTiming
    static void processOrder() {

        validateOrder();
        calculatePrice();
        saveOrder();
    }

    @MethodTiming
    static void validateOrder() {

        sleep(100);
    }

    @MethodTiming
    static void calculatePrice() {
```

```

        sleep(200);
    }

    @MethodTiming
    static void saveOrder() {

        sleep(300);
    }

    static void sleep(long millis) {

        try {
            Thread.sleep(millis);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

```

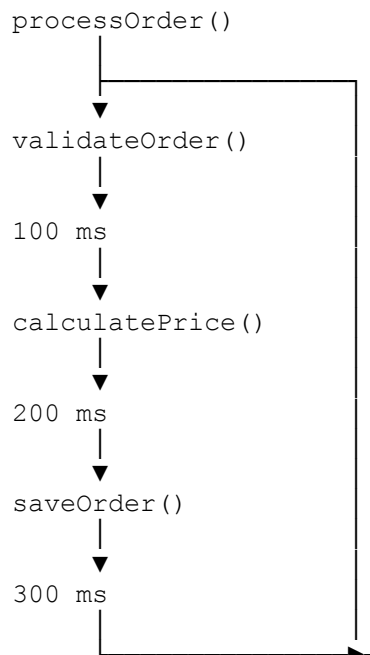
The annotation tells JFR that the method is a candidate for method timing/tracing.

5. Execution Flow

For:

`processOrder();`

the execution can be visualized as:



JFR can record timing information for the annotated methods.

Conceptually:

Method	Duration
<code>validateOrder()</code>	100 ms
<code>calculatePrice()</code>	200 ms
<code>saveOrder()</code>	300 ms
<code>processOrder()</code>	~600 ms

The exact recorded values will differ because of scheduling, JVM behavior, and measurement overhead.

6. Starting JFR

Compile the application normally:

```
javac JFRMethodTimingTracing.java
```

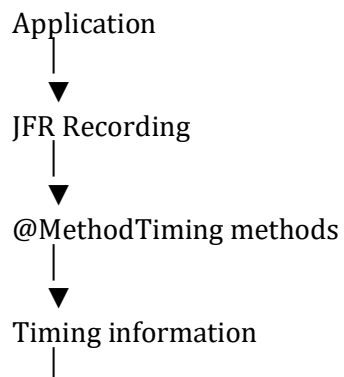
Then run it with JFR:

```
java -XX:StartFlightRecording=\
duration=10s,\
filename=method-timing.jfr \
JFRMethodTimingTracing
```

The recording is saved as:

method-timing.jfr

Conceptually:



▼
method-timing.jfr

7. Inspecting the Recording

You can inspect the recording with the JFR command-line tool:

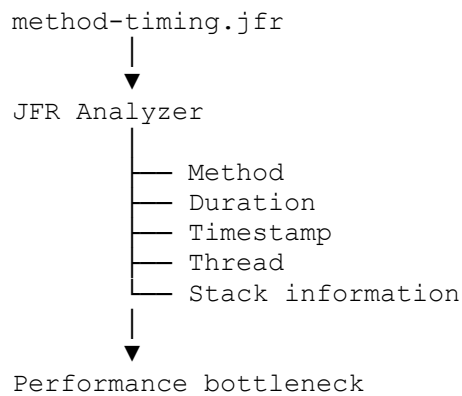
```
jfr summary method-timing.jfr
```

You can also print events:

```
jfr print method-timing.jfr
```

Or use **JDK Mission Control** for graphical analysis.

Conceptually:



8. Method Timing vs Manual Timing

Traditional manual timing

You might write:

```
long start = System.nanoTime();  
  
calculatePrice();  
  
long end = System.nanoTime();  
  
System.out.println(  
    "Duration = " + (end - start)  
);
```

This has several disadvantages:

- timing code becomes mixed with business logic,
- you have to maintain the timing statements,
- results aren't naturally integrated into JFR,
- it becomes cumbersome when many methods need measurement.

JFR method timing

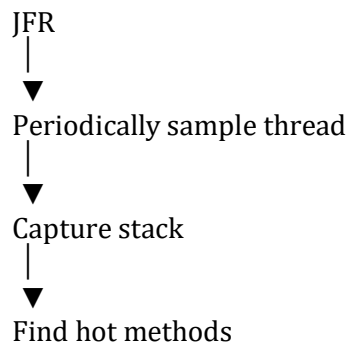
```
@MethodTiming
static void calculatePrice() {
    // business logic
}
```

The JVM/JFR infrastructure handles the profiling.

9. Method Timing vs JFR Execution Sampling

These are different JFR techniques.

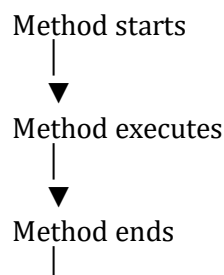
Execution sampling



This answers:

"Where is the application spending execution time?"

Method timing



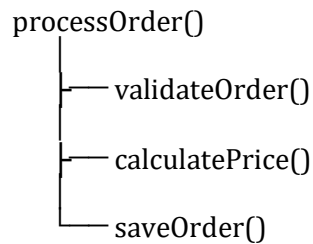
▼
Measure duration

This answers:

"How long did this particular method invocation take?"

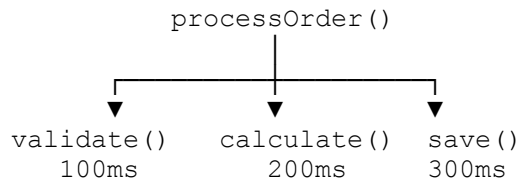
10. Tracing Call Relationships

The tracing aspect can be visualized as:



JFR can use method timing information to help understand these execution relationships.

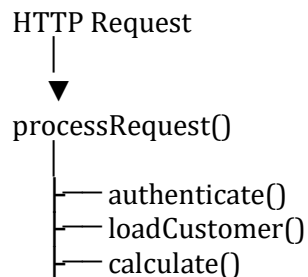
Conceptually:



This makes it easier to locate the expensive portion of a method's execution.

11. Practical Example

Imagine a web application:



└─ save()

You could annotate selected methods:

```
@MethodTiming
static void authenticate() {
    // ...
}

@MethodTiming
static void loadCustomer() {
    // ...
}

@MethodTiming
static void calculate() {
    // ...
}

@MethodTiming
static void save() {
    // ...
}
```

Then JFR can help answer:

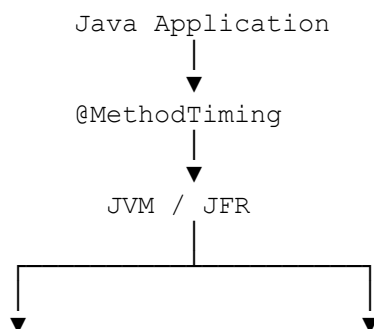
Which method is slowest?

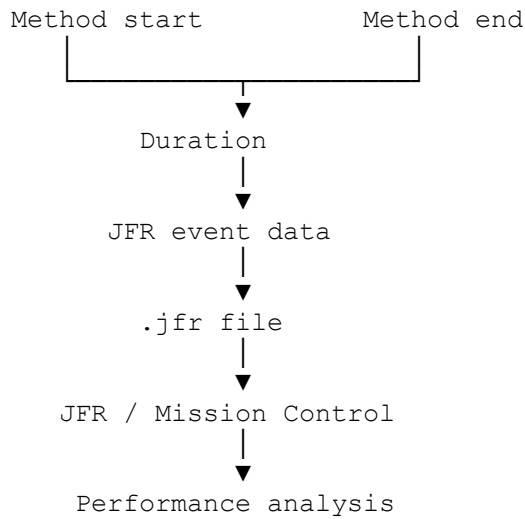


Investigate database operation

This is useful because you can instrument **specific methods** rather than attempting to trace every method in the application.

12. Conceptual Architecture



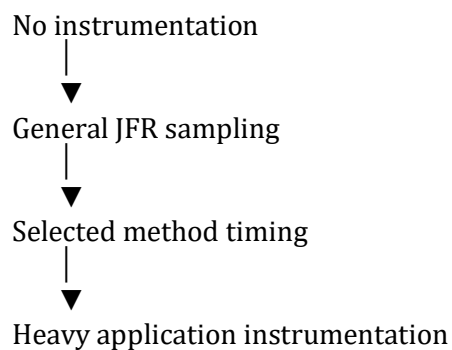


13. Why This Feature Is Useful

JFR Method Timing & Tracing is particularly useful for:

- diagnosing slow methods,
- application performance analysis,
- production troubleshooting,
- identifying expensive method calls,
- understanding selected call paths,
- investigating latency.

It provides a middle ground between:



14. Important Java 25 Catalog Entry

For your Java feature catalog:

Java Version : 25

Year : 2025
Feature : JFR Method Timing & Tracing
UniqueClassName : JFRMethodTimingTracing
JEP : 520
Status : Final

Key Takeaway

JFR Method Timing & Tracing (JEP 520) in Java 25 provides a lightweight JFR-based mechanism for timing selected Java methods. By marking methods for timing, developers can obtain method execution-duration information integrated into JFR recordings, making it easier to identify slow methods and investigate application performance without adding manual `System.nanoTime()` instrumentation throughout the code.

Java 25 — Generational Shenandoah

Java Version: 25
Year: 2025
Feature: Generational Shenandoah
UniqueClassName: GenerationalShenandoah
JEP: 521
Status: Final

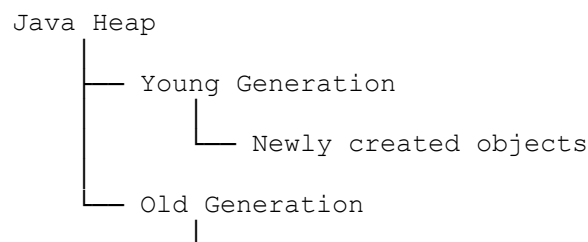
1. Definition

Generational Shenandoah makes **generational garbage collection** the standard operating mode of the **Shenandoah garbage collector** in Java 25.

The basic idea is based on the **generational hypothesis**:

Most newly created objects become unreachable relatively quickly, while objects that survive for a long time tend to remain useful for longer.

Instead of treating the entire Java heap uniformly, Generational Shenandoah divides the heap conceptually into:



└ Long-lived objects

The garbage collector can therefore focus collection effort on the **young generation**, where garbage is generally more abundant.

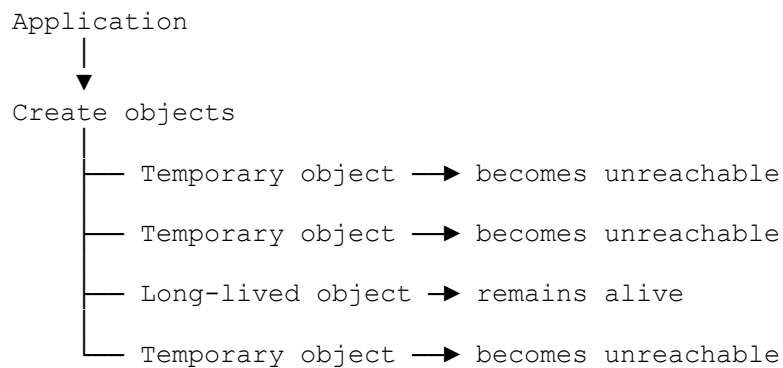
2. Why Generational Garbage Collection?

Consider a typical Java application:

```
Customer customer = new Customer();  
Order order = new Order();  
String message = new String("Hello");
```

Applications continuously allocate temporary objects.

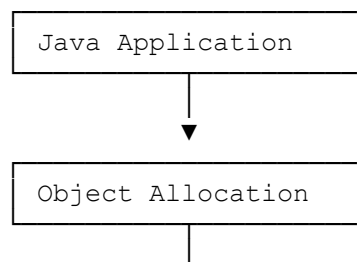
Conceptually:

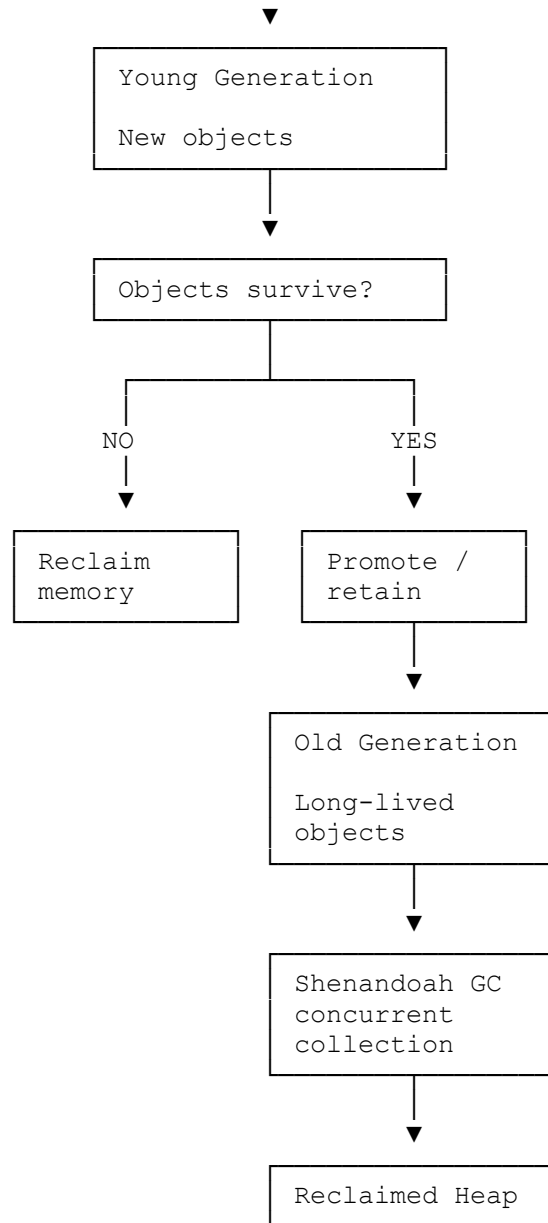


Most short-lived objects don't need to remain in the heap for a long time.

Generational collection exploits this behavior.

3. Flowchart





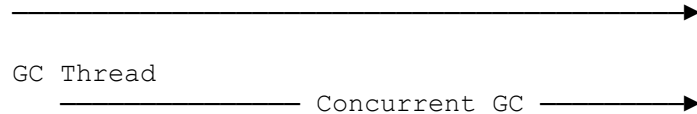
4. Shenandoah Before Generational Mode

Shenandoah is designed as a **low-pause garbage collector**.

Its major characteristic is that much of the GC work can occur concurrently with application execution.

Conceptually:

Application Thread

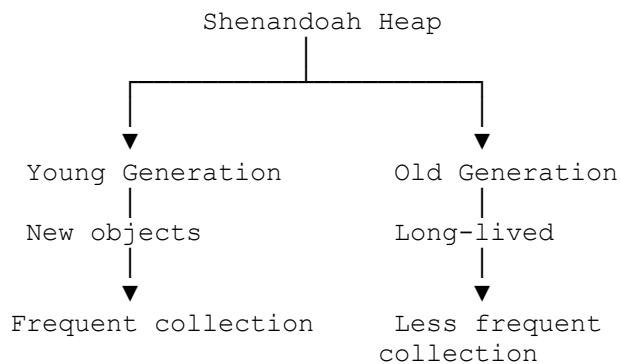


This is different from a collector that performs large amounts of GC work while stopping the application.

5. Generational Shenandoah

Java 25 makes the generational mode the standard Shenandoah configuration.

Conceptually:



The collector can concentrate on regions containing large amounts of recently allocated objects.

6. Java Code Example

There is **no new Java language syntax or Java API** for Generational Shenandoah.

Your Java application continues to create objects normally.

```
public class GenerationalShenandoah {  
    public static void main(String[] args) {  
        System.out.println(  
            "Starting Generational Shenandoah demo"  
        );  
        for (int i = 0; i < 1_000_000; i++) {  
            createTemporaryObjects();  
        }  
    }  
}
```

```

        if (i % 100_000 == 0) {
            System.out.println(
                "Processed: " + i
            );
        }
    }

    System.out.println("Finished");
}

static void createTemporaryObjects() {
    for (int i = 0; i < 100; i++) {
        String temporary =
            "Temporary-" + i;

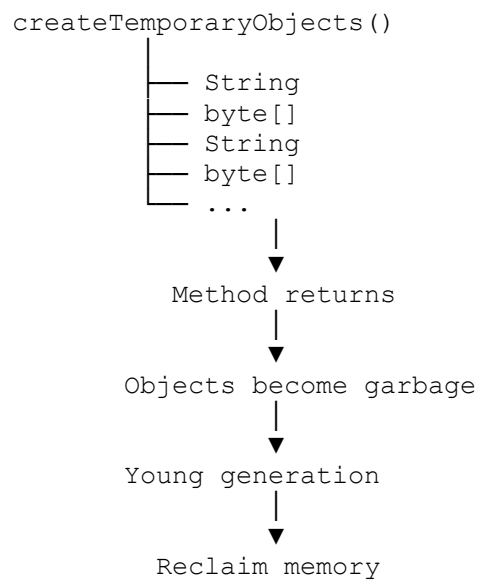
        byte[] buffer =
            new byte[1024];

        // Objects become unreachable after
        // this method returns.
    }
}
}

```

The important point is that this program creates a large number of **short-lived objects**.

Conceptually:



7. Running with Shenandoah

You can explicitly select Shenandoah using:

```
java -XX:+UseShenandoahGC GenerationalShenandoah
```

On Java 25, generational Shenandoah is the default Shenandoah mode, so this does not mean you are selecting the older non-generational mode.

You can verify the selected collector with:

```
java -Xlog:gc GenerationalShenandoah
```

For more detailed GC logging:

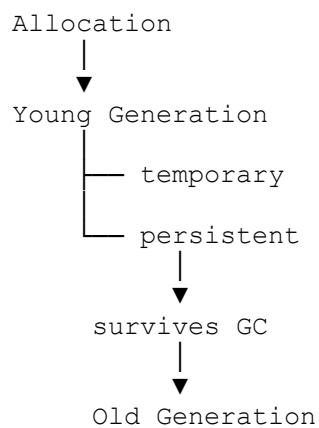
```
java -Xlog:gc*=info GenerationalShenandoah
```

8. Memory Lifecycle

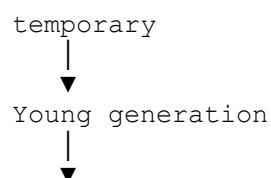
Consider these objects:

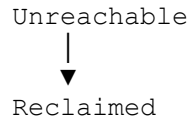
```
Object temporary = new Object();  
Object persistent = new Object();
```

Conceptually:



The temporary object may disappear quickly:

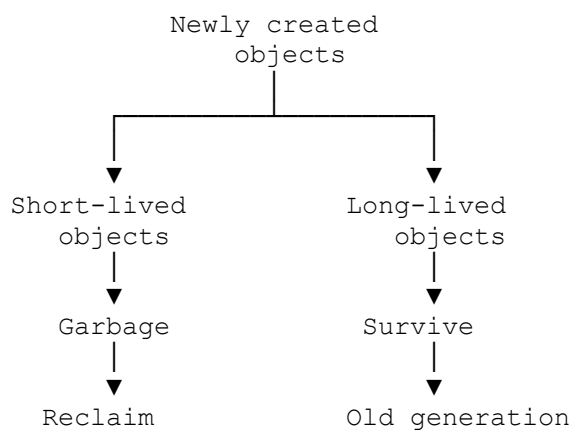




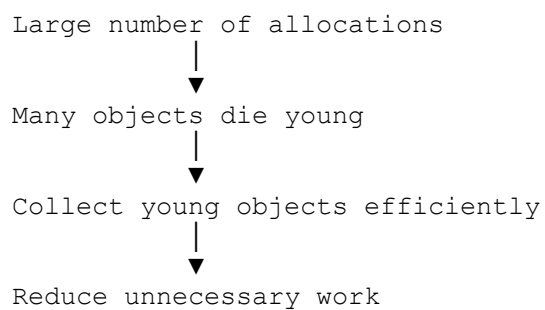
while a long-lived object can survive and become part of the old generation.

9. Generational Hypothesis

The central assumption can be illustrated as:

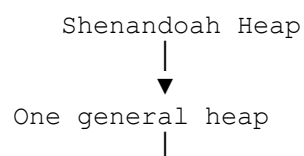


In many workloads:



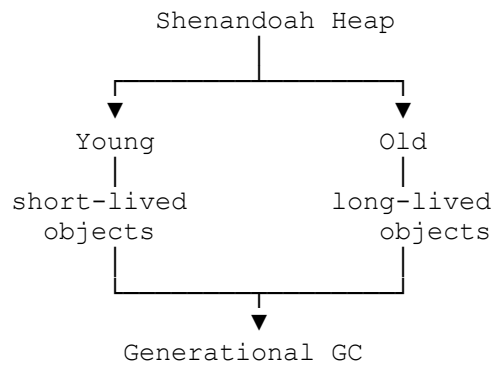
10. Generational vs Non-Generational Shenandoah

Non-generational model



▼
Concurrent GC

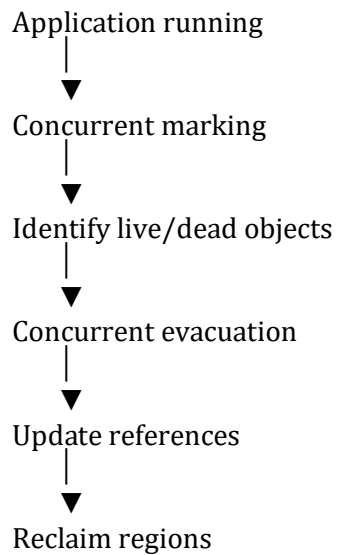
Generational model



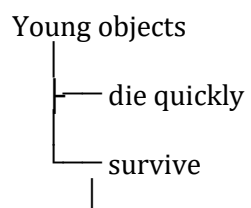
11. Important Shenandoah Concepts

Shenandoah is known for performing significant GC work concurrently.

A simplified lifecycle is:



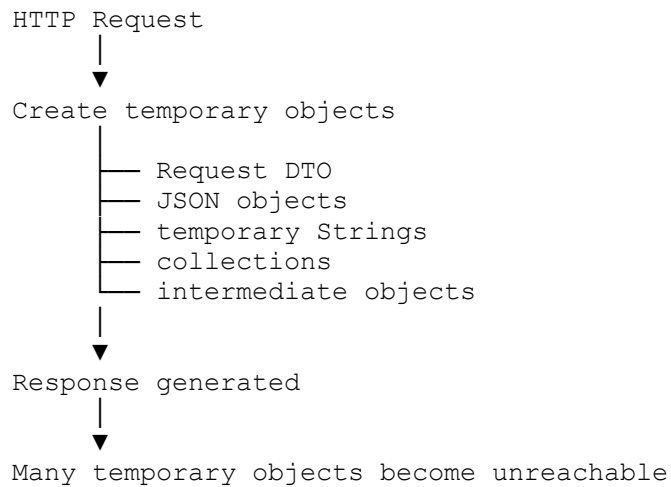
With generations, the collector can also reason about:



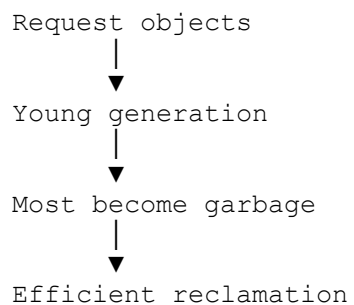
▼
Old objects

12. Example: Object-Heavy Application

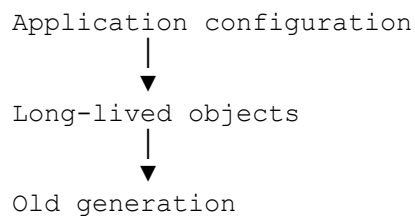
Imagine a web server processing requests:



Generational Shenandoah can exploit this pattern:



Meanwhile:



13. Advantages

Generational Shenandoah aims to provide:

Better efficiency

Focus GC work where garbage is concentrated.

Lower allocation overhead

Young objects can be reclaimed efficiently.

Good latency characteristics

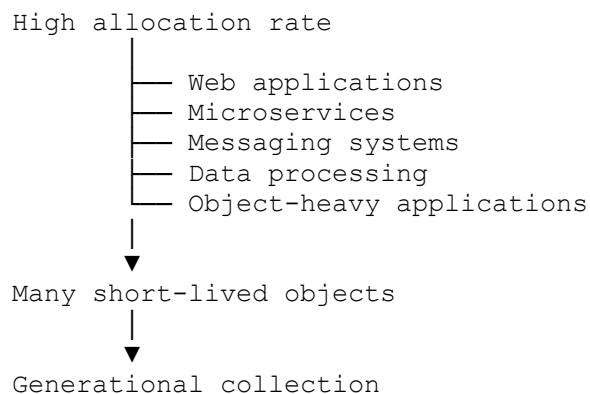
Shenandoah retains its emphasis on concurrent garbage collection and low pauses.

Better scalability for object-heavy workloads

Applications that allocate large numbers of short-lived objects can benefit particularly from generational collection.

14. When Is It Useful?

Generational Shenandoah can be particularly attractive for:



It is not automatically the best collector for every workload; GC selection should be based on actual latency, throughput, heap-size, and allocation characteristics.

15. Java 25 Catalog Entry

For your Java feature catalog:

Java Version	: 25
Year	: 2025
Feature	: Generational Shenandoah
UniqueClassName	: GenerationalShenandoah
JEP	: 521
Status	: Final

The important Java 25 change is that **Generational Shenandoah becomes the default mode for the Shenandoah garbage collector.**

Key Takeaway

Generational Shenandoah (JEP 521) in Java 25 makes generational collection the standard mode of Shenandoah. It divides the heap conceptually into young and old generations, allowing the collector to focus more efficiently on newly allocated, short-lived objects while retaining Shenandoah's concurrent, low-pause garbage-collection design.